PenTester Field Handbook

A Visual Study Companion — Pre-Security → Cyber Security 101 → SOC Level 1 → Advent of Cyber 2025

Welcome

A visual, cross-linked rewrite of your Obsidian vault — the journey from **Pre-Security** foundations through **Cyber Security 101**, **SOC Level 1**, and the hands-on **Advent of Cyber 2025** labs, distilled into diagrams, comparison tables, and highlighted key concepts.

How to use this handbook

- **Collapsible contents** — the sidebar is a three-level tree. Collapse or expand by **Part** (learning path), by **Topic** (chapter), or drill into **sub-topics**. Use **Expand all / Collapse all** at the top to flip the whole tree at once. On a phone, tap ☰.
- **Search** — type in the search box to filter the contents to matching topics; matches are highlighted and their chapters auto-expand.
- **Light / dark** — use the ◐ Theme button (top-right) to switch; your choice is remembered.
- **Colour-coded by path** — each Part has its own colour so you always know where you are:

Pre-Security foundations CS 101 core skills SOC L1 blue team Advent of Cyber hands-on labs Reference quick lookups

- **Cross-links** — within the text, links like this jump to related topics.
- **Nothing is lost** — anything trimmed from your original notes is preserved verbatim in The Noise at the end.

The callout boxes

Six kinds of highlighted box flag the important bits:

> 🔑 **KEY CONCEPT — THE BIG IDEA OF A SECTION.**
>
> The one thing to take away.

> 📘 **DEFINITION — A PRECISE TERM YOU SHOULD KNOW.**
>
> A term defined exactly.

> 🧪 **EXAMPLE — A WORKED, CONCRETE ILLUSTRATION.**
>
> A real command or scenario.

> 💡 **TIP — A SHORTCUT, MNEMONIC, OR BEST PRACTICE.**
>
> A faster or safer way.

> ⚠️ **WARNING — A SECURITY PITFALL OR SOMETHING ATTACKERS EXPLOIT.**
>
> Watch out for this.

> 🗒️ **NOTE — CONTEXT OR AN ASIDE WORTH REMEMBERING.**
>
> Background that helps.

What's inside

Part	Chapters	Covers
Pre-Security	1 – 8	Careers & CIA triad, networking, OSI, LAN, web, Linux, Windows, virtualization/cloud, data & encoding
CS 101	9 – 11	Command line (Bash/CMD/PowerShell), search skills, Active Directory
SOC L1	12 – 16	Attack vectors, SOC structure, SIEM/EDR/SOAR, frameworks, triage & metrics
Advent of Cyber	17 – 19	IDOR, XSS, SQLi, race conditions, cURL, malware/HTA, YARA, registry, RITA, Nmap, passwords, Docker, AWS, Modbus, phishing
Reference	20 – 22	Command reference & cheat-sheets, learning roadmap, The Noise

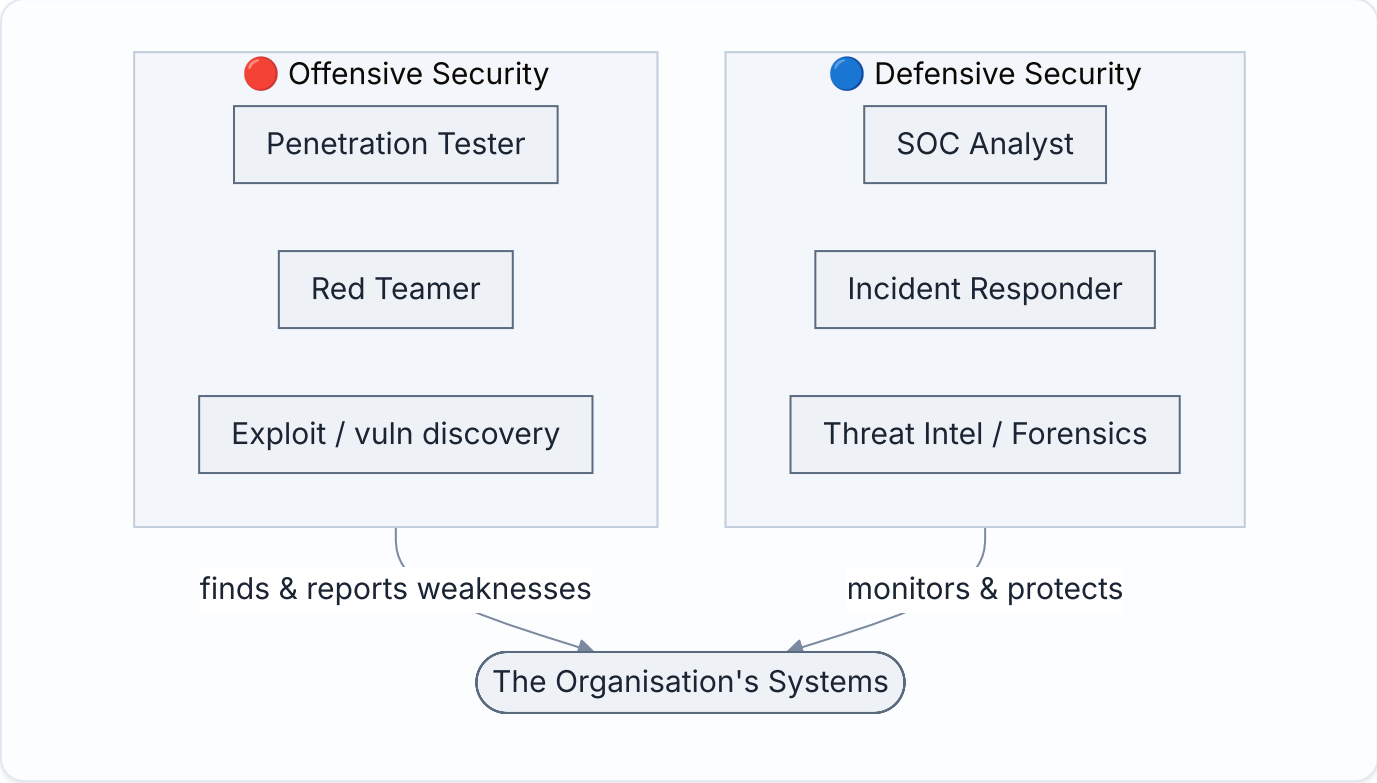
> 🗒️ **EDITABLE & YOURS**
>
> Every chapter is a plain Markdown file in the `src/` folder (Obsidian-compatible, with callouts and Mermaid diagrams). Edit any chapter and re-run the build script to regenerate this page and the PDF. Your original vault was not modified.

1 · Security Mindset, Careers & the CIA Triad

Offensive vs Defensive Security

Cyber security splits into two complementary mindsets. The offensive side *thinks like an attacker* to find weaknesses; the defensive side *monitors, detects and responds* to protect systems.

> 🔑 **THE CORE IDEA**
>
> *"To outsmart a hacker, you need to think like one."* Offensive security breaks into systems to find flaws before criminals do. Defensive security (**blue teaming**) builds and watches the walls.



	Offensive Security (Red)	Defensive Security (Blue)
Goal	Think like an attacker to find flaws first	Protect networks, detect intrusions, respond
Mindset	*"To outsmart a hacker, think like one."*	*"Defence in depth"* — layered controls
Typical work	Breaking in, exploiting bugs, finding loopholes	Monitoring, incident response, threat intel
Example tool	Gobuster brute-forcing hidden pages	SIEM correlating logs; IDS; firewalls

> 🧪 **YOUR FIRST HACK (THE FAKEBANK LAB)**
>
> `gobuster` brute-forces hidden web directories from a wordlist, revealing an unprotected `/bank-transfer` admin page — a real-world *access control* failure. `Status: 200` means the page exists; if an admin left it public, an attacker finds it — and an ethical hacker reports it before a criminal abuses it.

```
gobuster -u http://fakebank.thm -w wordlist.txt dir
# -u  target URL      -w  wordlist      dir  directory brute-force mode
# Status: 200 = page exists   301 = redirect
```

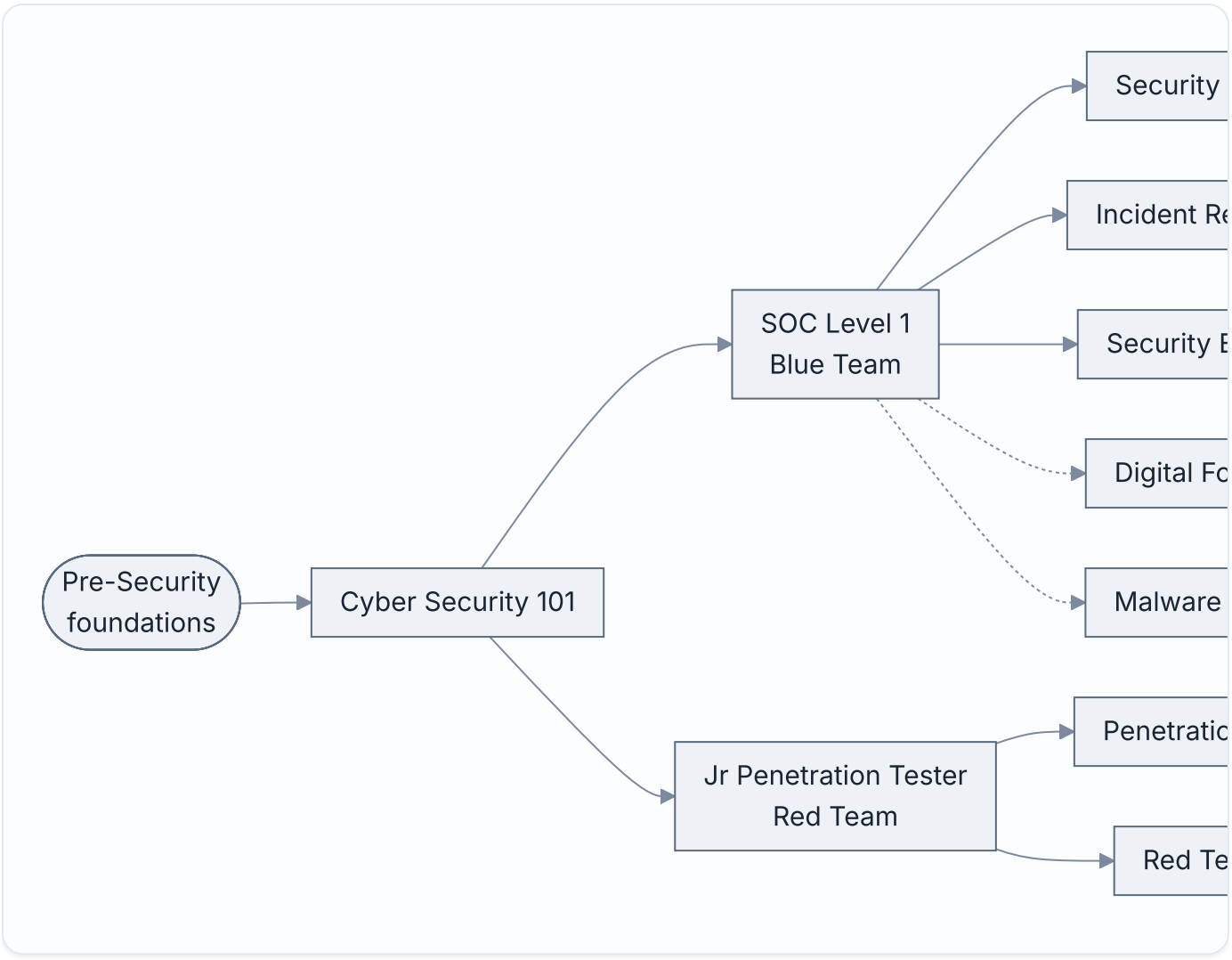
Defence in Depth

Organisations never rely on one control. They layer them like an onion or a castle's walls, so that if one fails, others still hold.

Defensive layer	Role
Employee training	Humans are the most-attacked surface; teach phishing awareness
IDS (Intrusion Detection System)	"Surveillance cameras" that alert on suspicious activity
Firewalls	"Security guards" deciding what traffic may enter/leave
Security policies	Rules (strong passwords, blocked sites) that shrink the attack surface

The five defensive responsibilities: **Monitoring & Detecting · Incident Response · Threat Intelligence · Vulnerability Management · Investigation & Analysis.**

The Cyber Careers Map



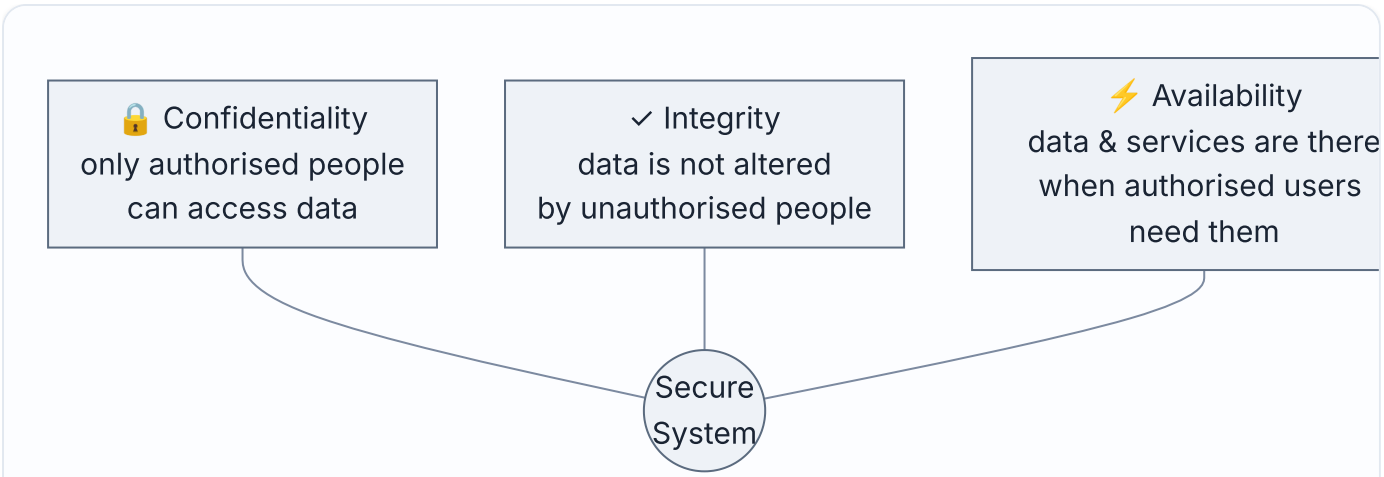
Role	One-line summary	Key metric / skill
Security Analyst	Maintains an organisation's data security; builds measures across teams	Reporting, risk analysis
Security Engineer	Designs & maintains controls, networks, systems	Threat & vuln data
Incident Responder	Mitigates attacks *while they unfold*	MTTD / MTTA / MTTR
Digital Forensics Examiner	Investigates incidents & crimes from digital evidence	Legal chain of custody
Malware Analyst	Reverse-engineers malicious programs	Assembly, C, static+dynamic analysis
Penetration Tester	Tests products for exploitable flaws (with permission)	Systemised hacking, reporting
Red Teamer	Emulates a real adversary to test detection & response	Stealth, persistence, evasion

📘 **INCIDENT-RESPONSE TIMING METRICS**

- **MTTD — Mean Time to Detect:** average time to *identify* a threat.
- **MTTA — Mean Time to Acknowledge:** time from alert to an analyst *taking action*.
- **MTTR — Mean Time to Respond:** time from alert to the breach being *stopped* (malware removal, password reset, restore).

The CIA Triad

Everything in cyber security protects three properties of data. This is the model you will return to constantly.



Pillar	Protects against	Achieved? — example
Confidentiality	Eavesdropping, data theft	Gmail creds on a sticky note → **No** · internal docs limited to staff who need them → **Yes**
Integrity	Unauthorised modification	Grades changed after lock → **No** · change via approved process → **Yes**
Availability	Downtime, DoS	Site offline in business hours → **No** · systems reachable during work hours → **Yes**

💡 **WHY IT MATTERS**

When you triage an alert or assess a risk, ask *which pillar is threatened*. Ransomware attacks **Availability** (and often Confidentiality); a defaced website attacks **Integrity**; a leaked database attacks **Confidentiality**.

2 · Networking Fundamentals

A network is simply *things connected*. In computing, anywhere from 2 to billions of devices communicate using shared rules (**protocols**).

Identifying Devices: IP & MAC

Every device has two identifiers — like a changeable *name* and a permanent *fingerprint*.

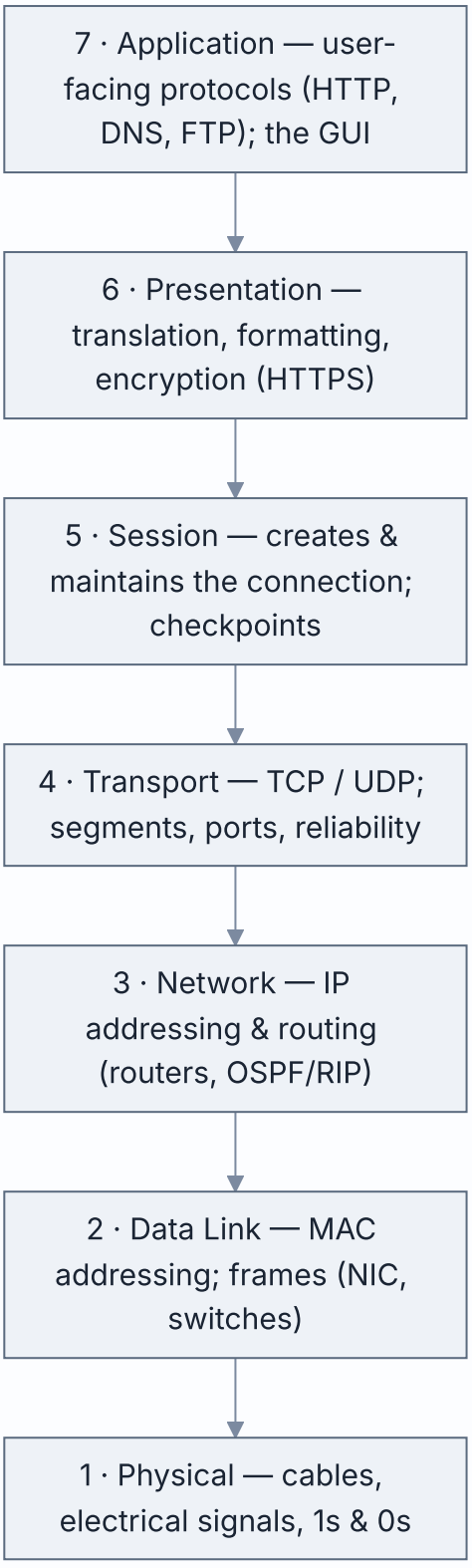
Identifier	Analogy	Changeable?	Format
IP address	Your name (can change)	Yes — reassigned over time	`192.168.1.77` (4 octets, IPv4)
MAC address	Your fingerprint / serial	Burnt in at factory (but *spoofable*)	`a4:c3:f0:85:ac:2d` (12 hex chars)

> 📘 **PRIVATE VS PUBLIC IP**
>
> A **private** IP identifies a device *within* a local network (e.g. `192.168.1.74`). A **public** IP identifies your whole network *on the Internet* and is issued by your ISP. Many devices share one public IP.

> 🔑 **IPV4 VS IPV6**
>
> IPv4 has 2^{32} ≈ 4.29 billion addresses — nearly exhausted. **IPv6** uses 2^{128} (≈ 340 trillion-plus) addresses and is more efficient. A MAC's first 6 hex chars identify the manufacturer; the last 6 are unique. MAC addresses can be **spoofed**, breaking security that trusts them (e.g. café Wi-Fi allow-lists).

The OSI Model

The **O**pen **S**ystems **I**nterconnection model is the 7-layer framework describing how all networked devices send, receive and interpret data. Each layer adds its own information — a process called **encapsulation**.



💡 **REMEMBER THE LAYERS**

All **P**eople **S**eem **T**o **N**eed **D**ata **P**rocessing (7→1). Layer 6 is the **translator** (and where encryption like HTTPS lives). Firewalls operate at **layers 3 & 4**.

Layer	Deals with	Devices / examples
7 Application	How the user interacts with data	Browsers, email clients, DNS
6 Presentation	Standardising & encrypting data	HTTPS/TLS, data formats
5 Session	Opening/closing/maintaining sessions	Checkpoints to resume lost data
4 Transport	TCP vs UDP, ports, error-checking	Port numbers
3 Network	Routing via **IP addresses**	Routers (Layer-3 devices)
2 Data Link	Physical **MAC** addressing	NIC, Layer-2 switches, frames
1 Physical	Raw bits on the wire	Ethernet cables

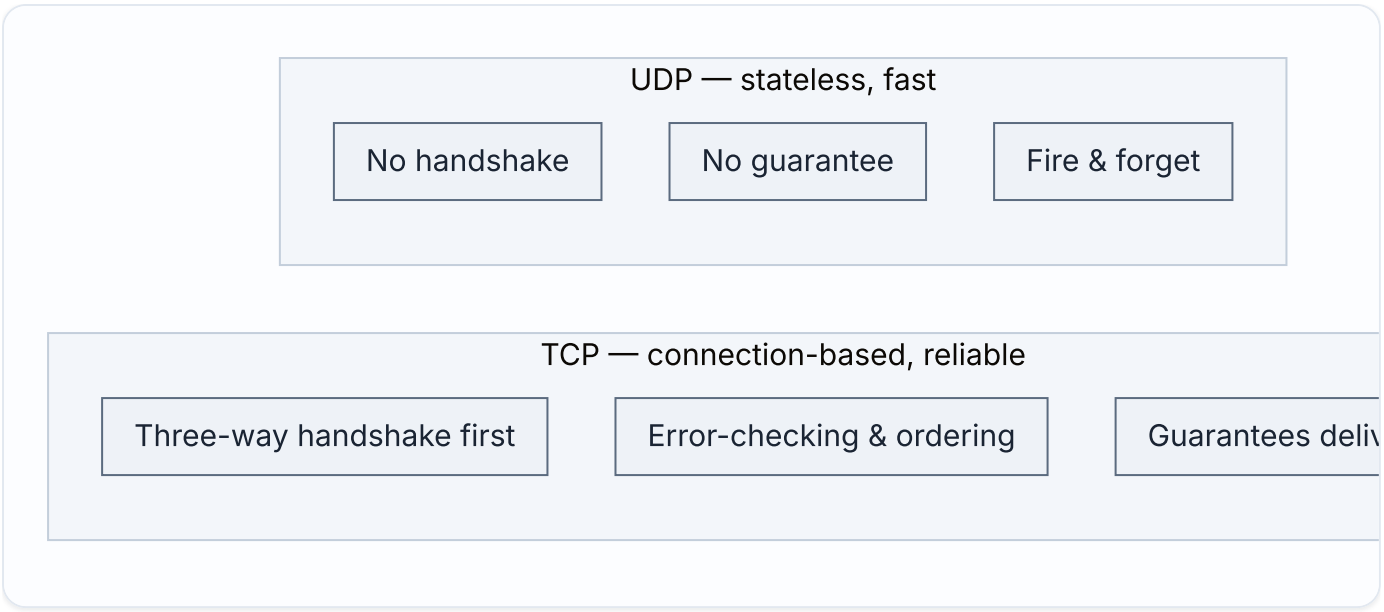
Packets & Frames

> 📘 **PACKET VS FRAME**
>
> A **packet** is Layer-3 data: an IP header + payload. A **frame** is Layer-2: it *encapsulates* the packet and adds MAC addresses. Think of mailing a letter — the **envelope is the frame**, the **letter inside is the packet**.

Data is split into small pieces so a network doesn't bottleneck. Key IP headers: **Time To Live** (expiry), **Checksum** (integrity), **Source Address**, **Destination Address**.

TCP vs UDP

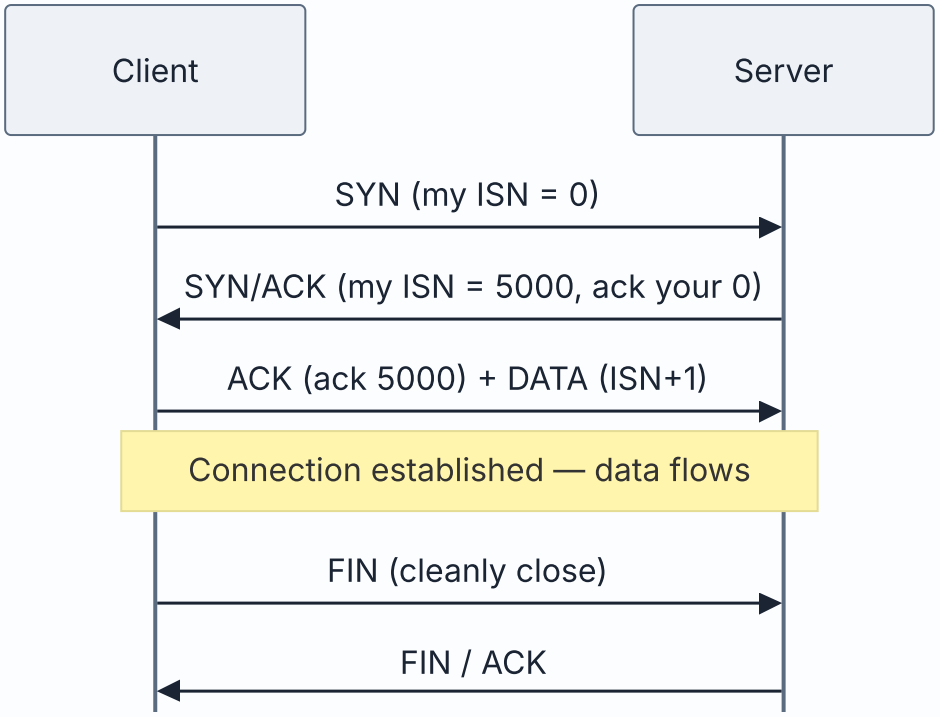
Layer 4 chooses between two transport protocols.



	TCP	**UDP**
Connection	Establishes one first (handshake)	None — stateless
Reliability	Guarantees accuracy & order	No guarantee; data may be lost
Speed	Slower (more processing)	Much faster
Use cases	File sharing, web, email	Video/voice streaming, ARP, DHCP

The TCP Three-Way Handshake

TCP must establish a connection *before* sending data, synchronising sequence numbers so pieces reassemble in order.



📘 **KEY TCP FLAGS**

SYN initiate & synchronise · **SYN/ACK** server acknowledges · **ACK** acknowledges received data · **DATA** carries the payload · **FIN** cleanly closes · **RST** abruptly ends (last resort / error).

Important TCP headers include **Source/Destination Port**, **Source/Destination IP**, **Sequence Number**, **Acknowledgement Number**, **Checksum** (integrity), **Data**, and **Flag**.

Ports

Ports are numbered endpoints (0–65535) that let a device run many services at once. Ports **0–1024** are "common" and standardised.

Port	Protocol	Purpose
21	**FTP**	File transfer (client-server)
22	**SSH**	Secure remote text login/management
80	**HTTP**	The World Wide Web (unencrypted)
443	**HTTPS**	HTTP with encryption
445	**SMB**	File & printer sharing
3389	**RDP**	Remote graphical desktop

> 💡 **NON-STANDARD PORTS**
>
> Standards are conventions, not rules — a web server *can* run on 8080. To reach it you must specify the port with a colon: `http://site.com:8080`.

Ping / ICMP

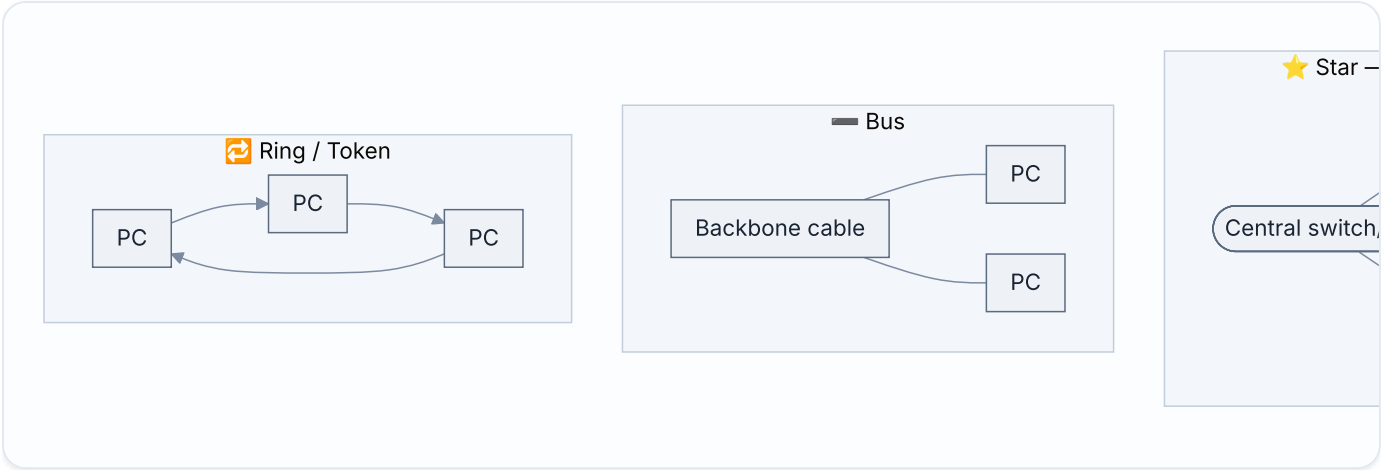
`ping` uses **ICMP** echo request/reply packets to test whether a host is reachable and measure round-trip time.

```
ping 192.168.1.254    # or ping tryhackme.com
```

3 · LANs, Devices & Network Extension

LAN Topologies

The *topology* is the design or shape of a network. Each has trade-offs.



Topology	Pros	Cons
Star	Scalable, reliable, easy to add devices	Costly (cabling + hardware); central device is a single point of failure
Bus	Cheap, simple	Backbone bottlenecks; one cable break kills it; hard to troubleshoot
Ring	Little cabling, less bottlenecking, easy fault isolation	Inefficient (data hops many devices); one break halts the ring

Switches & Routers

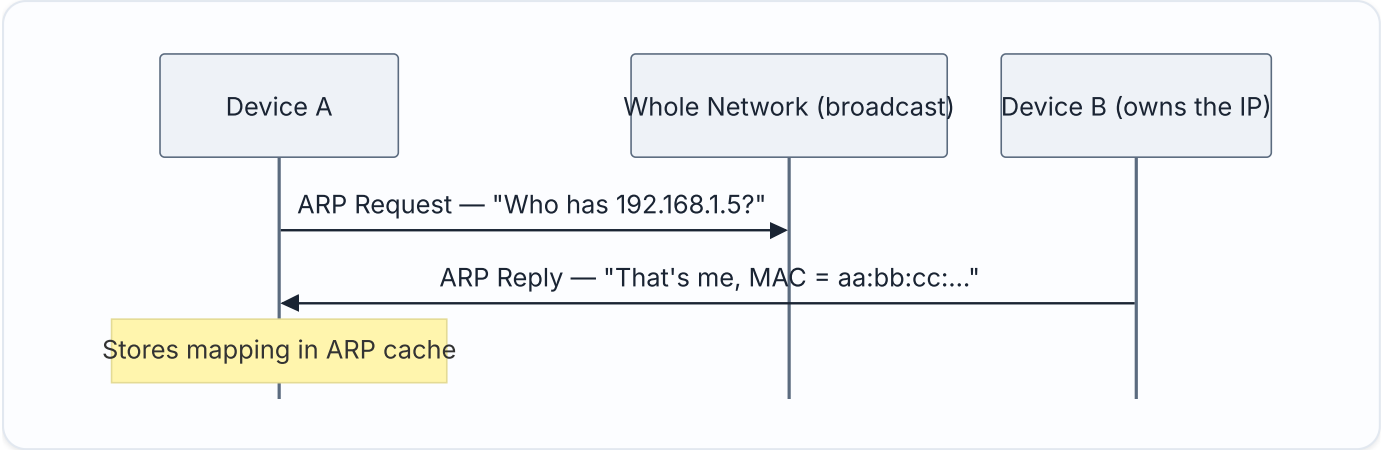
> 📘 **SWITCH VS ROUTER**
>
> A **switch** aggregates many devices (ports of 4–64) on one network and forwards *frames* to the correct device by MAC address (Layer 2) — far more efficient than a hub, which repeats to everyone. A **router** connects *different networks* and passes data between them by **routing** packets (Layer 3), choosing paths by shortest/most-reliable/fastest medium.

A **Layer-3 switch** can do some routing. **VLANs** (Virtual LANs) virtually split devices on one switch so, e.g., Sales and Accounting share Internet but cannot talk to each other — adding security

segmentation.

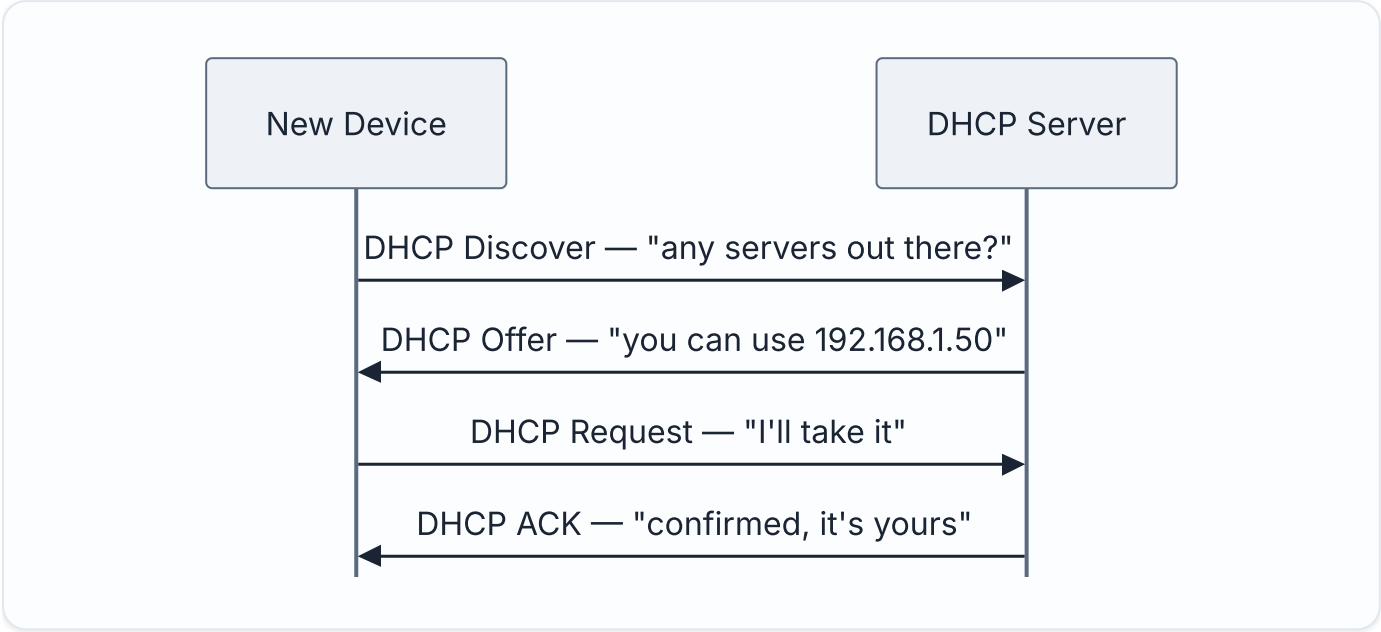
ARP — Address Resolution Protocol

ARP maps an **IP address → MAC address** so devices can actually deliver frames. Each device keeps an **ARP cache**.



DHCP

Dynamic **H**ost **C**onfiguration **P**rotocol automatically assigns IP addresses. The exchange is **DORA**:



Subnetting

Subnetting splits one network into smaller logical networks. A **subnet mask** (32 bits, each octet 0–255) decides how many hosts fit.

Type	Purpose	Example
Network Address	Identifies the network's start	`192.168.1.0`
Host Address	Identifies a device on the subnet	`192.168.1.100`
Default Gateway	Device that forwards traffic to *other* networks	`192.168.1.254` (first/last host)

> 💡 **WHY SUBNET?**
>
> **Efficiency, security, and control.** A café separates its public Wi-Fi subnet from the staff/till subnet while both still reach the Internet.

Extending the Network

Firewalls

A firewall is border security for a network — it permits or denies traffic by **source, destination, port, and protocol**, performing packet inspection. It operates at **OSI layers 3 & 4**.

Firewall type	Inspects	Trade-off
Stateful	The *entire connection* (dynamic)	Resource-heavy; blocks a whole device if a connection is bad
Stateless	*Individual packets* against static rules	Cheap & fast but "dumber"; great against DDoS from many hosts

VPNs

A **V**irtual **P**rivate **N**etwork creates an encrypted *tunnel* so devices on separate networks communicate securely as one private network.

🔑 **VPN BENEFITS**

Connects geographically-separate networks · **Privacy** (encryption defeats sniffing, useful on public Wi-Fi) · **Anonymity** (only as good as the provider's logging policy).

VPN technology	Notes
PPP	Handles auth + encryption via private key & certificate; *non-routable* by itself
PPTP	Lets PPP data leave the network; easy to set up but weakly encrypted
IPSec	Encrypts using the IP framework; harder to configure but strong & widely supported

Port Forwarding

Port forwarding (configured at the **router**) makes an internal service reachable from the Internet by opening a specific port and mapping the public IP to an internal host.

⚠️ **PORT FORWARDING ≠ FIREWALL**

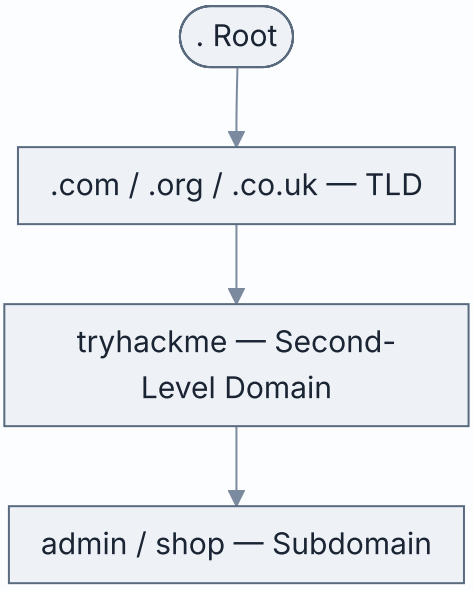
Port forwarding *opens* a port; a firewall *decides whether traffic may travel* through it, even when it's open.

4 · How the Web Works

DNS — Domain Name System

DNS translates human names (`tryhackme.com`) into IP addresses (`104.26.10.229`) so you don't memorise numbers.

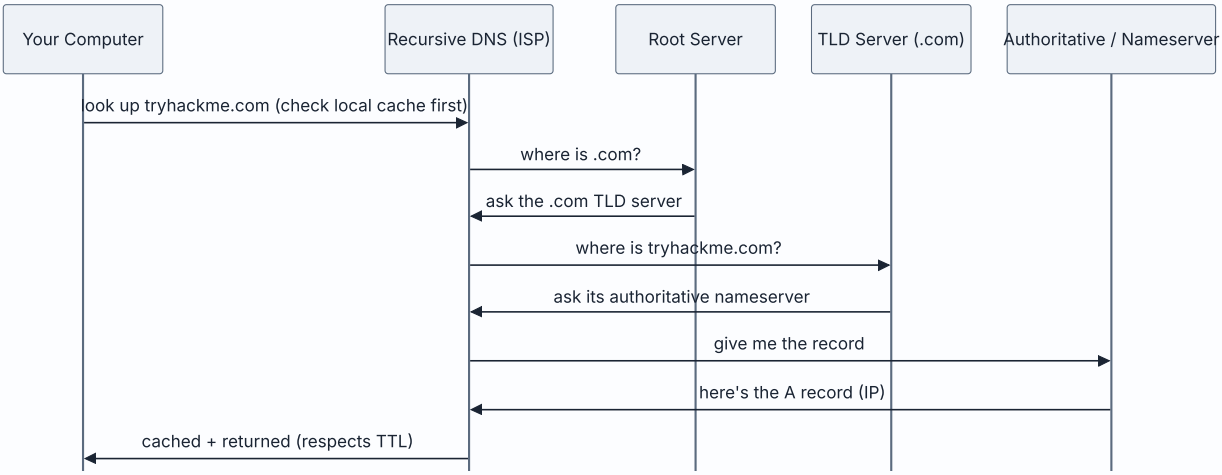
The Domain Hierarchy



📘 **DOMAIN NAME PARTS**

- **TLD** (Top-Level Domain): the rightmost part. *gTLD* (`.com` , `.org`) or *ccTLD* (`.co.uk` , `.ca`).
- **Second-Level Domain**: e.g. `tryhackme` — max 63 chars, only `a-z 0-9 -` .
- **Subdomain**: left of the SLD (`admin.tryhackme.com`), same rules, whole name ≤ 253 chars.

A DNS Request's Journey



DNS Record Types

Record	Resolves to	Example
A	IPv4 address	`104.26.10.229`
AAAA	IPv6 address	`2606:4700:20::681a:be5`
CNAME	Another domain name	`store.*` → `shops.shopify.com`
MX	Mail servers (with priority)	`alt1.aspmx.l.google.com`
TXT	Free text (SPF, domain verification)	anti-spam records

HTTP & HTTPS

> 📘 **HTTP VS HTTPS**
>
> **HTTP** (HyperText Transfer Protocol) is the rulebook for requesting web resources. **HTTPS** is the same but **encrypted** — stopping eavesdropping *and* proving you're talking to the genuine server.

URL Anatomy

```
http://user:password@tryhackme.com:80/view-room?id=1#task3
└┬┘   └──────┬──────┘ └─────┬──────┘└┬┘└──┬──┘└──┬──┘└┬┘
scheme     user:pass       host     port  path    query  fragment
```

Request / Response

```
GET / HTTP/1.1
Host: tryhackme.com
User-Agent: Mozilla/5.0 Firefox/87.0
```

```
HTTP/1.1 200 OK
Server: nginx/1.15.8
Content-Type: text/html
Content-Length: 98
```

HTTP Methods

Method	Intent
GET	Retrieve information
POST	Submit data / create records
PUT	Update information
DELETE	Remove records

Status Codes

Range	Meaning	Common codes
100–199	Informational	100 Continue
200–299	Success	**200** OK · 201 Created
300–399	Redirection	301 Moved Permanently · 302 Found
400–499	Client errors	400 Bad Request · 401 Not Authorised · 403 Forbidden · **404** Not Found · 405 Method Not Allowed
500–599	Server errors	500 Internal Error · 503 Service Unavailable

Headers & Cookies

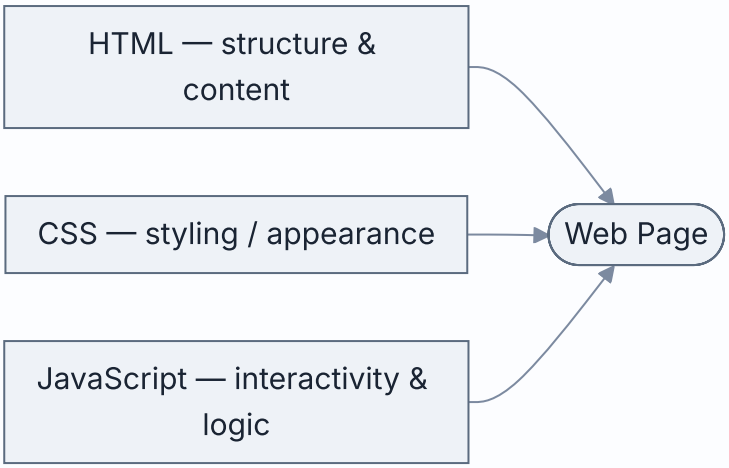
Request header	Tells the server...
Host	Which site (one server hosts many)
User-Agent	Your browser & version
Content-Length	How much data to expect
Cookie	Your stored session data

Response header	Tells the browser...
Set-Cookie	Data to store & resend each request
Cache-Control	How long to cache
Content-Type	Data type (HTML, image, JSON...)

📘 **COOKIES**

HTTP is **stateless** — it forgets you between requests. A **cookie** (saved from a `Set-Cookie` header) is resent on every request so the server remembers who you are. Auth cookies hold a **token** (a secret code), not your password.

How Websites Are Built



Language	Job
HTML	Building blocks (tags/elements) define structure: `<html><head><body><h1><p>`
CSS	Styling via `class` attributes
JavaScript	Interactivity; dynamically updates the page (`document.getElementById("demo").innerHTML = "…"`)

Front End vs Back End

- **Front End (client-side):** what your browser renders (HTML/CSS/JS).
- **Back End (server-side):** code you *can't* see (PHP, Python, Ruby, NodeJS…) that processes requests, talks to databases, and produces the HTML.

Two Classic Web Vulnerabilities

> ⚠️ **HTML INJECTION**
>
> When a site displays **unfiltered user input**, an attacker can inject their own HTML/JavaScript. The browser runs it — letting them control the page's look and behaviour. **Rule: never trust user input — sanitise it.**

⚠️ **SENSITIVE DATA EXPOSURE**

Developers sometimes leave credentials, hidden links or secrets in the **page source** or JavaScript. Always *View Page Source* when assessing a site. (These labs lead directly into the Advent of Cyber web labs: XSS, IDOR and more.)

Other Web Components

Component	Role
Web server (Apache, Nginx, IIS, NodeJS)	Serves content from a root dir (`/var/www/html` , `C:\inetpub\wwwroot`); uses **virtual hosts** to serve many sites
Load Balancer	Spreads traffic across servers (round-robin / weighted) + **health checks** for failover
CDN	Hosts static files on thousands of global servers; serves the nearest copy
Database	Stores data (MySQL, MSSQL, MongoDB, Postgres)
WAF (Web Application Firewall)	Sits in front of the server; blocks attacks, bots & abuse via **rate limiting**

📘 **STATIC VS DYNAMIC CONTENT**

Static content never changes (images, CSS). **Dynamic** content is generated per-request by back-end code (a blog homepage showing the latest post). Dynamic processing is where most web vulnerabilities live.

5 · Linux Fundamentals

Linux powers servers, security tools, Android, and most of the systems you'll attack or defend. Mastery of the **terminal** (text-based interface) is non-negotiable — it's faster, scriptable, and many tools are CLI-only.

Core Navigation & Files

Command	Name	Does
`pwd`	print working directory	Show where you are
`ls`	list	List directory contents (`ls -a` hidden, `ls -l` details, `ls -al` both)
`cd`	change directory	Move (`cd ..` up one level)
`cat`	concatenate	Output a file's contents
`echo`	—	Print text
`whoami`	—	Show current user
`find`	—	Search for files (`find -name *.txt` , `*` = wildcard)
`grep`	—	Search *inside* files for a pattern
`touch`	—	Create an empty file
`mkdir`	make directory	Create a folder
`cp` / `mv` / `rm`	copy / move / remove	Manage files (`rm -R` for folders)
`file`	—	Determine a file's real type

```
grep "81.143.211.90" access.log   # find every log line for an IP
find -name "*.txt"                 # every .txt in/below current dir
wc -l access.log                   # count lines
```

Shell Operators

Operator	Effect
`&`	Run a command in the **background**
`&&`	Run the next command **only if** the first succeeded
`>`	Redirect output to a file (**overwrite**)
`>>`	Redirect output to a file (**append**)

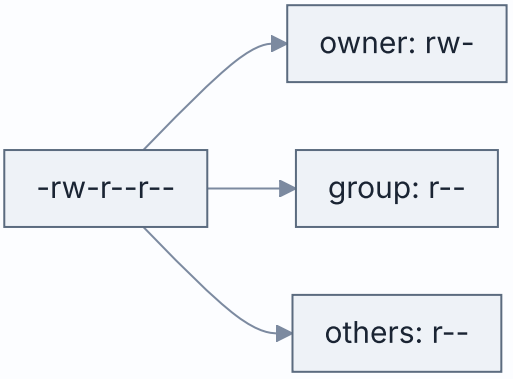
```
echo hey > welcome     # creates/overwrites "welcome" with "hey"
echo hello >> welcome  # appends "hello" on a new line
```

Flags, Switches & the Manual

Most commands accept **flags** (`-a`), have a `--help` option, and a full **man page**: `man ls` . The manual is your first stop — see also Search Skills.

Permissions & Users

`ls -l` shows permission strings like `-rw-r--r--` : **r**ead, **w**rite, e**x**ecute — split across **owner / group / others**.



📘 **SWITCHING USERS**

`su user2` switches user; `su -l user2` starts a *login* shell inheriting that user's environment and home directory. Privilege escalation often means moving toward **root** (the all-powerful user) via `sudo su`.

Filesystem Layout

Directory	Holds
`/etc`	System config — includes `passwd`, `shadow` (hashed passwords, SHA-512), `sudoers`
`/var`	Variable data — logs live in `/var/log`
`/root`	The root user's home
`/tmp`	Temporary, volatile (cleared on reboot); **world-writable** — handy for staging attack scripts

Useful Utilities

Command	Use
`wget <url>`	Download a file over HTTP
`scp src user@ip:/dest`	Securely copy files over SSH
`python3 -m http.server`	Instantly serve current dir on port 8000
`ssh user@ip`	Encrypted remote shell (see below)

🔑 **SSH**

Secure **Sh**ell encrypts a remote terminal session. Everything you type is encrypted in transit and decrypted on the remote host. The in-browser TryHackMe machines use this exact protocol. *When typing an SSH password you see no stars or dots — that's normal.*

Processes

Programs running on the machine. The kernel manages them; each has a **PID**.

Command	Use
`ps` / `ps aux`	List processes (aux = all users + system)
`top`	Real-time process stats
`kill <PID>`	Send a signal to end a process

> 📘 **KILL SIGNALS**
>
> **SIGTERM** — end, allowing cleanup · **SIGKILL** — end immediately, no cleanup · **SIGSTOP** — pause/suspend.

Process **0** boots the system (**systemd** on Ubuntu); everything else is a *child* of it. Manage services with `systemctl start|stop|enable|disable apache2`. **Foreground/background:** `&` or `Ctrl+Z` to background, `fg` to foreground.

Automation — Cron

`cron` runs scheduled tasks. A **crontab** line has 6 fields:

```
MIN  HOUR  DOM  MON  DOW  CMD
0    */12  *    *    *    cp -R /home/cmnatic/Documents /var/backups/
```

`crontab -e` to edit · `*` is a wildcard ("any"). (Online generators like crontab-generator.org help.)

Package Management

Software is installed from **apt repositories**. Add a trusted third-party repo with its **GPG key**, drop a `.list` file in `/etc/apt/sources.list.d/`, then `apt update` and `apt install <pkg>`. Remove with `apt remove`.

Logs & Text Editors

- **Logs** live in `/var/log` (Apache access/error, `fail2ban`, UFW firewall) — vital for monitoring health and security.
- **Nano** is the beginner-friendly editor (`^` = Ctrl, `^X` to exit). **VIM** is more powerful (syntax highlighting, customisable) but steeper.

6 · Windows Fundamentals

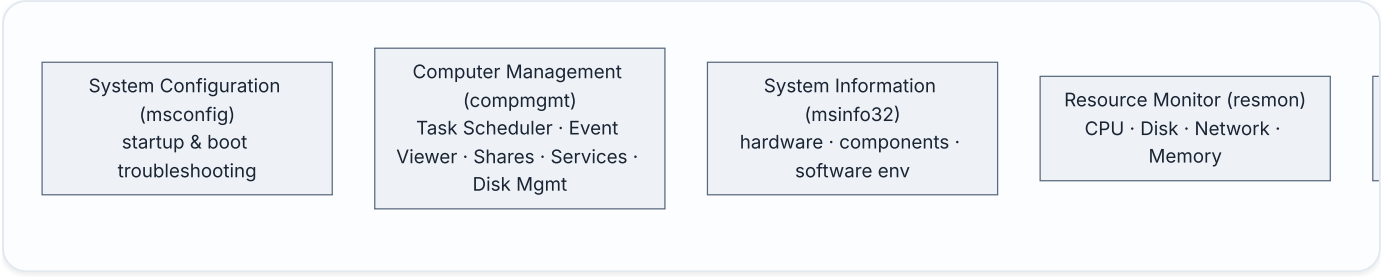
Windows dominates home and corporate networks, making it the biggest attack target. These notes cover its filesystem, admin tools, and built-in security.

The File System — NTFS

Modern Windows uses **NTFS** (New Technology File System), a *journaling* filesystem that can self-repair using a log. It replaced FAT16/FAT32 and HPFS (FAT still appears on USB sticks/SD cards).

NTFS feature	Notes
Large files	Supports files > 4 GB
Permissions	Full control · Modify · Read & Execute · List folder contents · Read · Write (set via *Properties → Security*)
Compression & Encryption	Folder/file compression; **EFS** encryption
Alternate Data Streams (ADS)	Every file has a `$DATA` stream; extra hidden streams can store data — **abused by malware to hide payloads**; also used to tag Internet-downloaded files

Key Admin Tools



Tool (run command)	Purpose
System Configuration (`msconfig`)	Diagnose startup issues (General/Boot/Services/Startup/Tools). *Needs admin; not a startup manager — use Task Manager for that.*
Computer Management (`compmgmt`)	Hub for **Task Scheduler** (run tasks automatically), **Event Viewer** (audit trail of events), **Shared Folders** (incl. default `C$` , `ADMIN$`), **Services**, **Disk Management**, **Perfmon**, **Device Manager**
System Information (`msinfo32`)	Hardware resources, components, software environment, **environment variables**
Resource Monitor (`resmon`)	Real-time per-process CPU/disk/network/memory

📘 **SERVICES & STARTUP TYPES**

A **service** is a special application that runs in the background. Its **Startup type** (Automatic / Manual / Disabled) decides when it runs. Event Viewer logs five event types and standard logs under *Windows Logs*.

💡 **ENVIRONMENT VARIABLES**

Store OS/program info — e.g. `WINDIR` holds the Windows install path. View via *Advanced System Settings → Environment Variables*.

Windows Security Features

Windows Update

Security fixes and patches arrive on **Patch Tuesday** (2nd Tuesday monthly); critical fixes can come sooner. `control /name Microsoft.WindowsUpdate` to open.

Windows Defender Firewall

Three **profiles**:

Profile	Used for
Domain	Networks where the host can authenticate to a domain controller
Private	Home / trusted networks
Public	Untrusted (coffee-shop Wi-Fi, airports) — the default

Open advanced rules with `WF.msc`. *Leave the firewall on unless you're 100% sure.*

Virus & Threat Protection (Defender AV)

- **Exclusions** — skip files/folders in scans (reduces false positives, but excluded items could hide threats).
- **Ransomware protection** — *Controlled folder access* (requires real-time protection on).
- On-demand scan: right-click → *Scan with Microsoft Defender*.

Volume Shadow Copy Service (VSS)

VSS creates consistent **shadow copies** (restore points) on protected drives. You can create restore points, perform system restore, configure or delete them.

> ⚠️ **MALWARE & SHADOW COPIES**
>
> Ransomware authors know about VSS and **delete shadow copies** so victims can't recover without paying. This is a classic indicator in ransomware incidents.

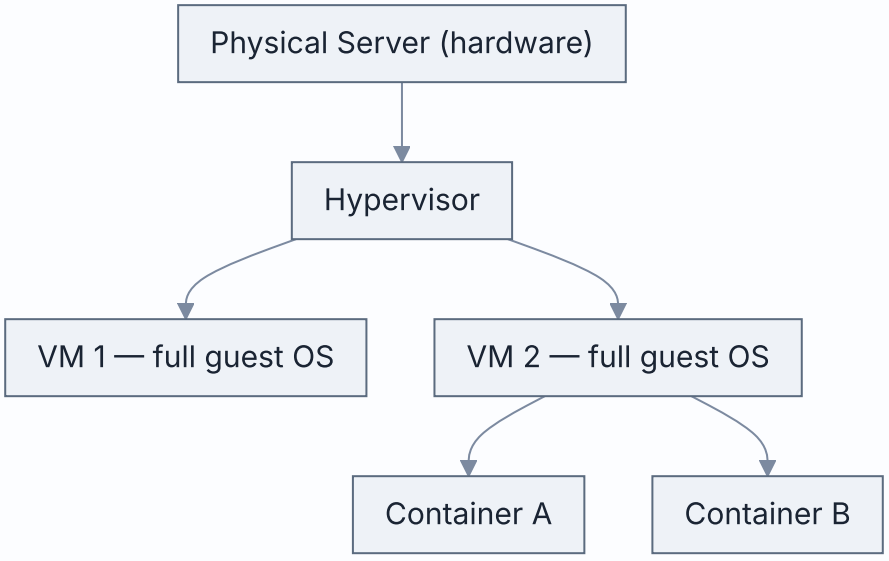
7 · Virtualization & Cloud

Virtualization

Before virtualization the rule was *"one server = one application"* — expensive, wasteful (5–20 % utilisation), slow to deploy, hard to scale. Virtualization lets **multiple applications safely share one physical server**.

🔑 **THE BUILDING ANALOGY**

- **Building** = the physical server
- **Apartments** = virtual machines (VMs)
- **Tenants** = applications / operating systems
- **Building manager** = the **hypervisor** that divides it safely



Concept	What it is
Hypervisor	Software that creates & manages VMs, allocating CPU/RAM/storage and keeping them isolated
Type 1	Runs *directly on hardware* — fast, for servers/data centres
Type 2	Runs *inside an existing OS* (VirtualBox, VMware Workstation) — easy, for learning/testing/malware labs
Virtual Machine	A whole virtual computer with its own OS, fully isolated
Container	Lightweight, isolated environment for *one app* that **shares the host kernel** — starts instantly, uses fewer resources, but must match the host OS type
Container image	Pre-packed template used to create containers (Docker)

> 💡 **MALWARE TESTING SAFETY**
>
> Use an **isolated VM** (often a different guest OS than your host) to test malicious files so the host isn't infected — Type 2 hypervisors are ideal. This ties into Malware Analysis and Containers.

VM vs Container in one line: a VM is the *full apartment* (max isolation, heavier); a container is a *room inside it* (lightweight, fast, scalable).

Cloud Computing

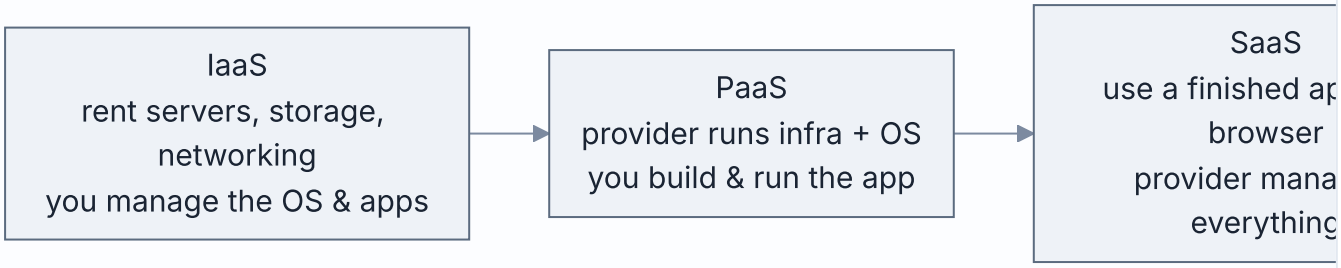
The cloud runs apps on provider infrastructure instead of your own hardware, built on virtualization & containers.

> 🔑 **BENEFITS**
>
> **Scalability · On-demand self-service · Pay only for what you use · Security · High availability · Global access.**

Deployment types

Type	Who uses it	Why
Public Cloud	Startups, global apps	Affordable, easy to scale, no infra to manage
Private Cloud	Banks, healthcare, government	More control, compliance for sensitive data
Hybrid Cloud	E-commerce	Keep sensitive data private, scale publicly on demand

Service models — the "renting a home" analogy



Model	You manage	Example
IaaS	OS + apps (rent raw compute)	AWS **EC2** virtual machines
PaaS	Just your application	App platforms
SaaS	Nothing — just use it	Gmail, Zoom

> 📘 **AWS TERMS FROM THE LAB**
>
> **EC2** = a virtual computer in the cloud (has CPU/RAM, runs apps). **Instance type** (`t2` , `t3` , `m5`) sets its power/cost — bigger = more power + higher cost. Stopping unused instances immediately cuts billing. (This foundation leads into AWS Security.)

Major vendors: AWS (leader), Microsoft Azure, Google Cloud (GCP), Alibaba Cloud, IBM Cloud, Oracle Cloud. Netflix, Spotify and Instagram run on the cloud to scale globally and focus on product over hardware.

8 · Data Representation, Encoding & Programming

Number Systems

Computers understand two states (low/high voltage) → **binary**. We group bits for readability.

System	Base	Digits	Groups
Decimal	10	0–9	everyday
Binary	2	0, 1	1 bit
Hexadecimal	16	0–F	4 bits per digit
Octal	8	0–7	3 bits per digit

> 📘 **BIT & BYTE**
>
> A **bit** is one binary digit (0/1). A **byte** (a.k.a. **octet**) is 8 bits and represents 256 states.

Binary → decimal (positional, powers of 2):

$$1001 = 1\times2^3 + 0\times2^2 + 0\times2^1 + 1\times2^0 = 8 + 0 + 0 + 1 = 9$$

Dec	Hex	Binary
9	`9`	`1001`
10	`A`	`1010`
12	`C`	`1100`
15	`F`	`1111`

Representing Colour

Each pixel mixes **Red, Green, Blue**. With 1 bit each you get $2^3 = 8$ colours; with **8 bits (256 levels) each** you get $256^3 \approx 16$ million. A colour is therefore **3 bytes (24 bits)**, written as 6 hex digits — e.g. green `A3EA2A` instead of `10100011 11101010 00101010`.

Character Encoding

> 📘 **REPRESENTATION VS ENCODING**
>
> **Representation** = data living as bits/numbers in memory. **Encoding** = the agreed mapping between numbers and meanings (which number is "A").

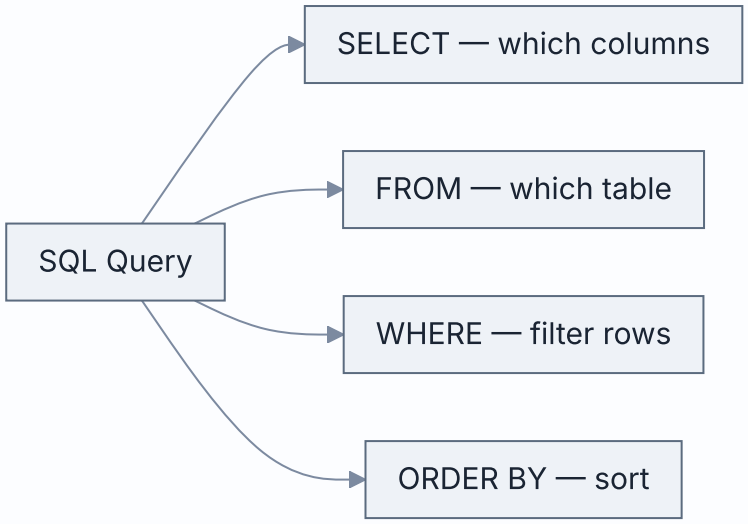
Encoding	Covers	Notes
ASCII (1963)	English letters, digits, punctuation (0–127)	7-bit; `A` =65/ `0x41` , `a` =97/ `0x61`
ISO-8859-1 / -2	Western / Central-European letters	Extended single-byte sets
Unicode	*Every* writing system (~157 k chars)	Code points like `U+0041` (A), `U+03A9` (Ω), `U+1F525` (🔥)
UTF-8	Unicode in **1–4 bytes**	Most common on the web
UTF-16 / UTF-32	2–4 / fixed 4 bytes	Common in Windows / simplest-but-wasteful

> 💡 **WHY GIBBERISH HAPPENS**
>
> If a file saved as one encoding is read as another, non-English characters display wrong — the classic cause of mojibake. Sender and receiver **must agree on the encoding**. This underpins decoding work in CyberChef and Obfuscation.

"TryHackMe" in ASCII binary: `01010100 01110010 01111001 01001000 01100001 01100011 01101011 01001101 01100101`.

Databases & SQL

A **database** stores information in **tables** (like spreadsheets): **columns** are fields (types of info), **rows** are records.



Keyword	Does
`SELECT *`	Choose all columns (`*` = all)
`FROM Orders`	Pick the table
`WHERE drink = 'Coffee'`	Filter matching rows
`ORDER BY price DESC`	Sort (DESC = high→low)

```
SELECT * FROM Orders WHERE drink = 'Coffee' ORDER BY price DESC;
```

> ⚠️ **SQL & SECURITY**
>
> A query reads data; it doesn't change it. But if user input is dropped unsanitised into a query, an attacker can manipulate it — **SQL injection** (seen in the AI-in-Security and web labs). Always use parameterised queries.

Programming Essentials (JS & Python)

Both notes build the same "guess the number" game, illustrating three universals:

Concept	JavaScript	Python
Variable (changeable)	`let guess = 0;`	`guess = 0`
Constant (fixed)	`const secret = …;`	— (`secret = …`)
Conditional	`if / else if / else`	`if / elif / else`
Loop	`while (guess !== secret) {…}`	`while guess != secret:`
Random	`Math.floor(Math.random()*20)+1`	`random.randint(1, 20)`
Input	`await rl.question(...)`	`input(...)`
Output	`console.log(...)`	`print(...)`

```
if guess < 1 or guess > 20:
    print("Out of range.")
elif guess < secret:
    print("Too low, try again.")
elif guess > secret:
    print("Too high, try again.")
else:
    print("You got it!")
```

🔑 **WHY THIS MATTERS FOR SECURITY**

Reading and lightly modifying scripts is a core skill — you'll analyse attacker PowerShell/VBScript, write YARA conditions, and automate attacks with Bash loops. You don't need to write apps from scratch, but you **must** be able to *look at code and explain what it does*.

9 · Command-Line Mastery

GUIs are intuitive; CLIs are **faster, lighter, scriptable, and remote-friendly** (SSH). This chapter covers the three shells you'll live in.

Linux Shells

> 📘 **WHAT IS A SHELL?**
>
> A shell interprets your typed commands. Most Linux distros default to **Bash**. The restaurant analogy: ordering from a menu (GUI) vs cooking it yourself (CLI) — the CLI gives full control.

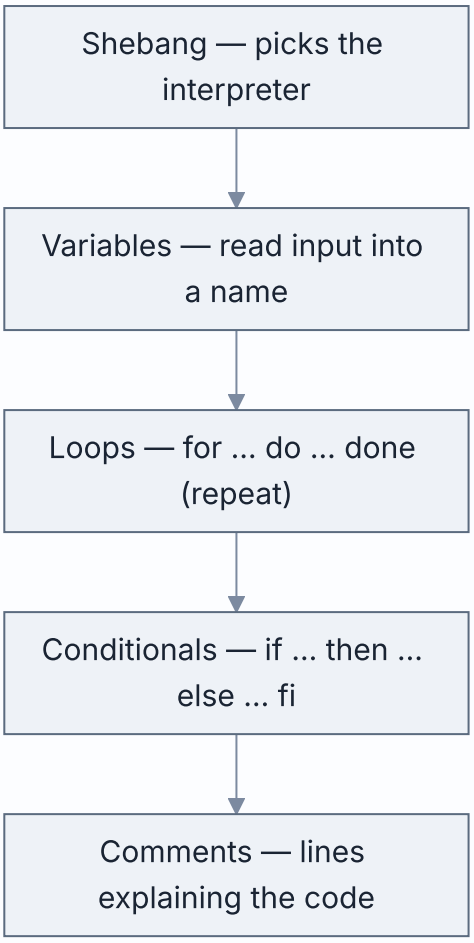
Shell	Full name	Standout features
Bash	Bourne Again Shell	Default everywhere; tab completion, history, strong scripting
Fish	Friendly Interactive Shell	Auto-suggestions, spell-correct, syntax highlighting built-in; beginner-friendly
Zsh	Z Shell	Advanced completion, `oh-my-zsh` themes/plugins, highly customisable

`echo $SHELL` shows your shell · `cat /etc/shells` lists installed shells · `chsh -s /usr/bin/zsh` changes default.

Shell Scripting

A script is a reusable set of commands in a `.sh` file.

```
#!/bin/bash                 # shebang: which interpreter to use
echo "Hey, what's your name?"
read name                   # take input
echo "Welcome, $name"       # $name = variable
```



Make it executable and run: `chmod +x script.sh` then `./script.sh` (the `./` tells the shell to run *this* file).

Windows Command Prompt (cmd.exe)

Task	Command
OS version	`ver`
Full system info	`systeminfo`
Env / path	`set`
Help / clear	`help` , `cls` , `command /?`
List dir	`dir` (`/a` hidden, `/s` subdirs), `tree`
Create / remove dir	`mkdir` , `rmdir`
View / copy / move / delete file	`type` , `copy` , `move` , `del` / `erase`

Networking: `ipconfig` (`/all` for DNS/DHCP) · `ping` · `tracert` (trace route) · `nslookup` · `netstat` (`-abon` : all, binaries, owner PID, numeric). **Processes:** `tasklist` (filter with `/FI "imagename eq sshd.exe"`) · `taskkill /PID <pid>` . Pipe long output through `more` (`driverquery | more`).

Windows PowerShell

> 🔑 **THE POWER IN POWERSHELL**
>
> Unlike text-based cmd, PowerShell passes **objects** (data + properties + methods) between commands. Cmdlets follow a `Verb-Noun` convention (`Get-Content` , `Set-Location`), built on .NET; cross-platform since **PowerShell Core** (2016).

Cmdlet	Equivalent	Does
`Get-ChildItem`	`dir` / `ls`	List files
`Set-Location`	`cd`	Change directory
`New-Item` / `Remove-Item`	create / `del` + `rmdir`	Create / delete
`Copy-Item` / `Move-Item`	`copy` / `move`	Copy / move
`Get-Content`	`type` / `cat`	Read a file
`Get-Command` / `Get-Help` / `Get-Alias`	—	Discover cmdlets, docs, shortcuts

Piping, filtering & sorting

```
Get-ChildItem | Sort-Object Length -Descending | Select-Object -First 1
Get-ChildItem | Where-Object -Property "Extension" -eq ".txt"
Select-String -Path ".\file.txt" -Pattern "hat"   # grep / findstr
```

Comparison operators: `-eq -ne -gt -ge -lt -le -like` .

Real-time system analysis (blue-team gold)

Cmdlet	Reveals
`Get-ComputerInfo`	Full system snapshot
`Get-LocalUser`	Local accounts
`Get-Process` / `Get-Service`	Running processes / services
`Get-NetTCPConnection`	Active TCP connections (C2 hunting)
`Get-NetIPConfiguration` / `Get-NetIPAddress`	Network config
`Get-FileHash`	File hash (integrity / malware ID)
`Get-Item -Path x -Stream *`	View **Alternate Data Streams**

> 💡 **WHY ANALYSTS LOVE POWERSHELL**
>
> `Invoke-Command -ScriptBlock { … }` runs commands on remote hosts — powering automation for incident response, threat hunting, and system administration. Red teams also use it for enumeration and remote execution, which is exactly why defenders watch for obfuscated/encoded PowerShell (see Web Attack Forensics).

> 📘 **THE MODERN `SS` COMMAND**
>
> On Linux, `netstat` is deprecated in favour of **`ss`** (socket statistics, in `iproute2`) — faster, talks directly to the kernel. Map: `netstat -a` → `ss -a`, `-t` → `ss -t` (TCP), `-l` → `ss -l` (listening), `-ltnp` → `ss -ltnp`. Routing table → `ip route`; interface stats → `ip -s link`.

10 · Search Skills & Vulnerability Research

Knowing *how to find answers* is itself a core skill. This chapter covers evaluating sources, search operators, specialised engines, and vulnerability databases.

Evaluating Search Results

Anyone can publish anything online, so weigh information by:

- **Source** — reputable, authoritative author/org?
- **Evidence & reasoning** — backed by hard facts and logic?
- **Objectivity & bias** — impartial, multiple perspectives?
- **Corroboration** — confirmed by multiple independent reliable sources?

> 📘 **SNAKE OIL**
>
> A cryptographic method/product that's bogus or ineffective. Red flags: *"unbreakable"/"100 % secure"* claims, secret **proprietary algorithms** (instead of vetted public standards like AES), vague "military-grade" marketing, and weak implementation.

Search Engine Operators

Harness advanced searches (Google / Bing / DuckDuckGo):

Operator	Effect	Example
`"exact phrase"`	Exact match	`"passive reconnaissance"`
`site:`	Limit to a domain	`site:tryhackme.com success`
`-`	Exclude a term	`pyramids -giza`
`filetype:`	Find a file type	`filetype:pdf cyber`

Specialised Search Engines



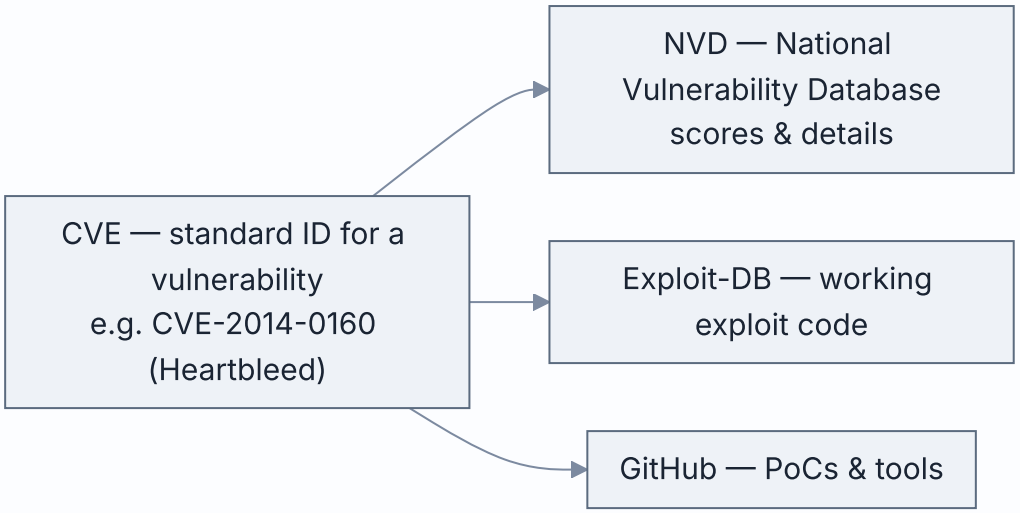
Engine	Finds
Shodan	Specific device/server types & versions exposed online (ICS, IoT)
Censys	Hosts, websites, and certificates
VirusTotal	Multi-engine virus scan of files/URLs + community notes
Have I Been Pwned (HIBP)	Whether an email appears in a data breach

Technical Documentation

Official docs are the most accurate source:

- **Linux man pages** — `man ip` (press `q` to quit); also searchable online.
- **Microsoft Learn** — official Windows/command docs (`ipconfig` , etc.).
- **Product docs** — Snort, Apache, etc.

Vulnerabilities & Exploits



📘 **CVE**

Common **V**ulnerabilities and **E**xposures — a dictionary giving each flaw a standard identifier, maintained by **MITRE**. Look them up at cve.org or the **NVD** (nvd.nist.gov).

⚠️ **EXPLOIT CODE — ETHICS FIRST**

The **Exploit Database**, GitHub PoCs, and similar hold working exploit code. Only run exploits against systems you are **explicitly authorised** to test. (This mindset carries into every offensive lab in Part IV.)

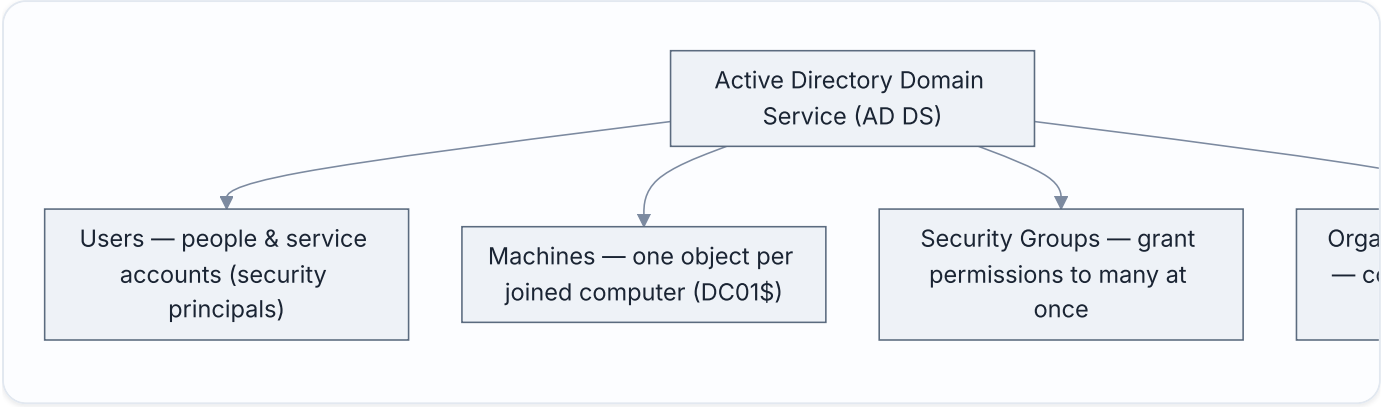
11 · Windows Domains & Active Directory

Managing five computers individually is fine; managing **157 computers and 320 users across four offices** is not. A **Windows domain** centralises administration — and Active Directory is the heart of it.

Windows Domains & AD DS

> 🔑 **WHY DOMAINS EXIST**
>
> A **Windows domain** is a group of users and computers under one business's administration. Main advantages: **centralised identity management** (configure all users from one place) and **managing security policies** (apply settings network-wide). You've met this at school/uni — one login works on every campus machine.

Active Directory Domain Services (AD DS) is the catalogue holding every **object** on the network.



Object	Notes
Users	**Security principals** (can authenticate & hold privileges). Represent *people* or *services* (IIS, MSSQL).
Machines	Created per joined computer; name ends in `$` (e.g. `DC01$`); local admin on their own host; 120-char auto-rotating password.
Security Groups	Assign rights to whole groups. Defaults: *Domain Admins, Server Operators, Backup Operators, Account Operators, Domain Users, Domain Computers, Domain Controllers*.
OUs	Organise objects (often mirroring departments: IT, Sales…) to apply policy. Default containers: Builtin, Computers, Domain Controllers, Users, Managed Service Accounts.

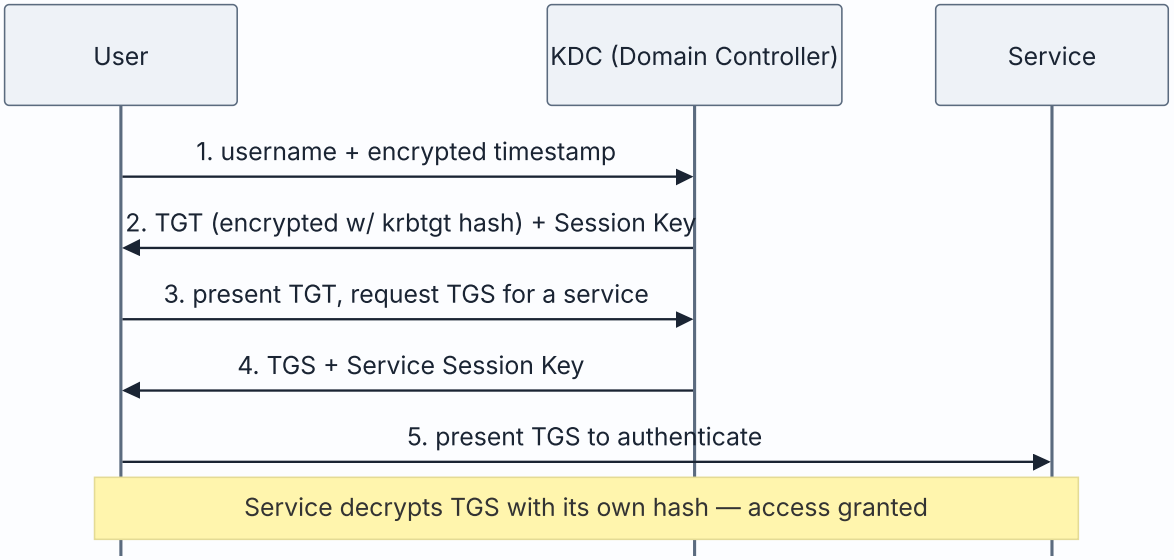
📘 **OUS VS SECURITY GROUPS**

OUs *apply policies* to users/computers (one object lives in one OU). **Security Groups** *grant permissions over resources* (an object can be in many groups). Different jobs entirely.

Authentication: Kerberos & NetNTLM

All domain credentials live on the **Domain Controllers**. When you authenticate to a service, it asks the DC to verify you — via one of two protocols.

Kerberos (modern default)



📘 **KERBEROS TICKETS**

- **TGT** (Ticket Granting Ticket) — proof of initial auth; encrypted with the **krbtgt** account hash.
- **TGS** (Ticket Granting Service) — grants access to one specific service; encrypted with the **service owner hash**.
- **KDC** (Key Distribution Center) — the DC service that issues tickets.

NetNTLM (legacy challenge-response)

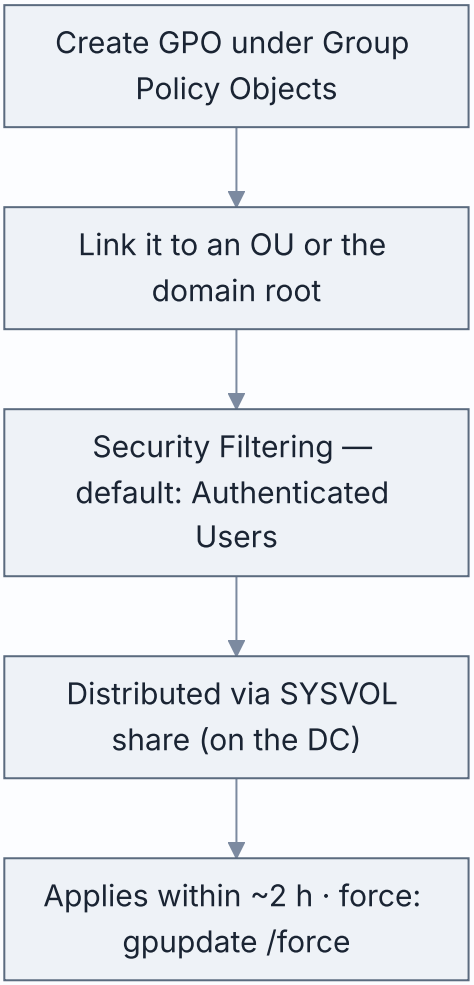
Kept for compatibility. The client proves it knows the password **without sending it**:

1. Client requests access → 2. Server sends a random **challenge** → 3. Client returns challenge + NTLM hash response → 4. Server forwards to DC → 5. DC recomputes & compares → 6. Result returned.

> 💡 **WHY THIS MATTERS OFFENSIVELY**
>
> Kerberos and NetNTLM are the backbone of countless AD attacks (Pass-the-Hash, Kerberoasting). The hash is **never transmitted directly** — attacks target the tickets and hashes instead. Pass-the-Hash detection appears in the Pyramid of Pain.

Group Policy Objects (GPOs)

A **GPO** is a collection of settings linked to OUs (or the whole domain) to push configuration — for *users* or *computers*.



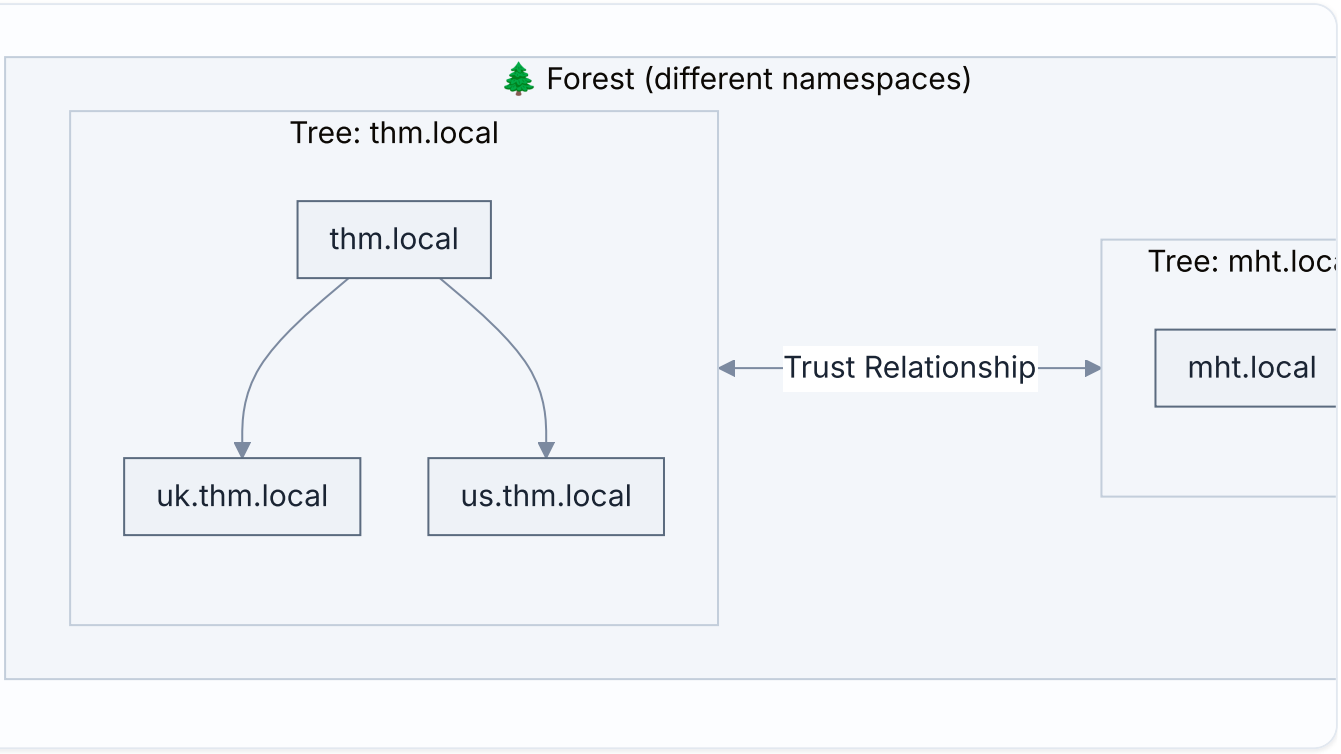
Worked examples from the lab:

Goal	Policy	Linked to
Block non-IT from Control Panel	*Prohibit Access to Control Panel and PC settings* (User Config)	Marketing, Management, Sales OUs
Auto-lock idle screens after 5 min	*Machine inactivity limit* (Computer Config)	Root domain (inherited by child OUs; user-only OUs ignore computer settings)

Manage via **Group Policy Management**; GPOs sync through the `SYSVOL` network share.

Trees, Forests & Trusts

As companies grow, one domain isn't enough.



Concept	Meaning
Tree	Multiple domains sharing one namespace (`thm.local` → `uk.thm.local` , `us.thm.local`)
Forest	Multiple trees with *different* namespaces joined (after a merger: `thm.local` + `mht.local`)
Enterprise Admins	Group with admin rights across *all* domains in an enterprise
Trust Relationship	Lets a user in one domain access resources in another

📘 **TRUST DIRECTION**

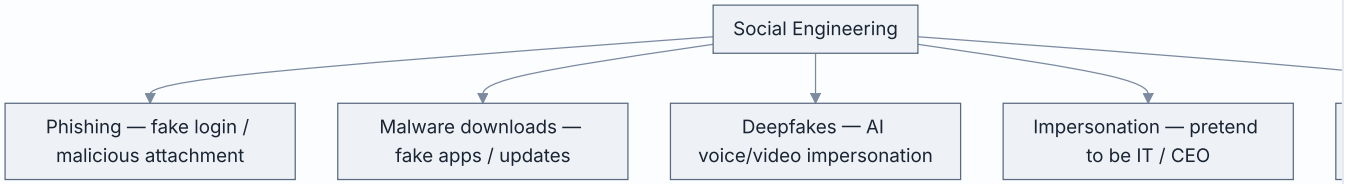
A **one-way trust**: if *AAA trusts BBB*, a user on *BBB* can access resources on *AAA* — the trust direction is **opposite** to the access direction. Joining domains in a tree/forest creates **two-way trusts** by default. A trust *allows* authorisation; it doesn't automatically *grant* access.

12 · Attack Vectors: Humans & Systems

Every attack needs a way in. Broadly, adversaries target **people** or **systems** — usually to gain **access**, then steal data, deploy ransomware, or destroy information.

Humans as an Attack Vector

> 🔑 **WHY ATTACK HUMANS?**
>
> *"Breach through the gate, or just ask the gatekeeper to open the door."* Humans provide **access** — to mailboxes, websites, databases. All human-targeted attacks rely on **social engineering**: manipulating psychology, not technology. Two ingredients: appear **trustworthy**, and trigger an **emotion** (urgency, fear, curiosity).



Breached target	Attacker's next step
HR manager's Google account	Steal & sell the employee database
Wealthy individual runs malware	Hijack their web-banking session
IT administrator's VPN account	Reach the heart of the corporate network

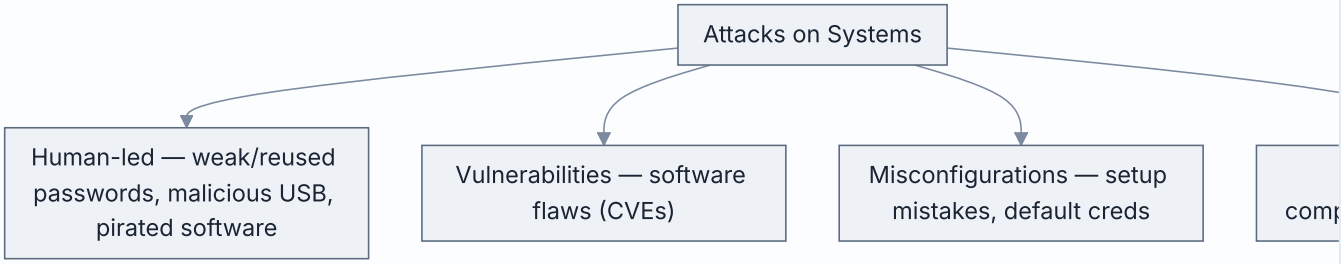
Defending humans

Mitigation	How it helps
Anti-phishing solution	Blocks phishing before users see it
Antivirus / EDR	Stops users running malware
"Trust but verify"	Verify suspicious "CEO/IT" requests via another channel
Security awareness training	Teach phishing detection + simulations

> 💡 **THE SOC ANALYST'S JOB**
>
> You can't stop every attempt — your job is to **detect and investigate**. Detection is the theme of the rest of Part III.

Systems as an Attack Vector

> 📘 **WHAT IS A SYSTEM?**
>
> Where data lives and runs: a physical server, a VM, or a cloud platform (Microsoft 365). Breaching one can mean data theft, ransomware, or defacement.



Vulnerabilities

Every piece of software has flaws; **40,000+ CVEs** were published in 2024. Once public, a CVE starts a race: attackers build exploits, defenders deploy **patches**.

> 📘 **RESPONDING TO A CVE**
>
> The answer is always **a patch** from the vendor. Before one exists (zero-day), buy time by: restricting access to trusted IPs, applying vendor workarounds, and blocking known attack patterns on **IPS/WAF**. Examples: Shellshock, EternalBlue (2017), PrintNightmare (2021), Follina.

Misconfigurations

Not a software bug — a **setup mistake** by the IT team (weak passwords like `1111`, exposed databases, misconfigured AWS). Often spotted only *after* exploitation.

Proactive defence	What it does
Penetration testing	Ethical hackers simulate attacks & report flaws
Vulnerability scans	Detect default passwords / outdated software
Configuration audits	Match systems to best practices (CIS benchmarks)

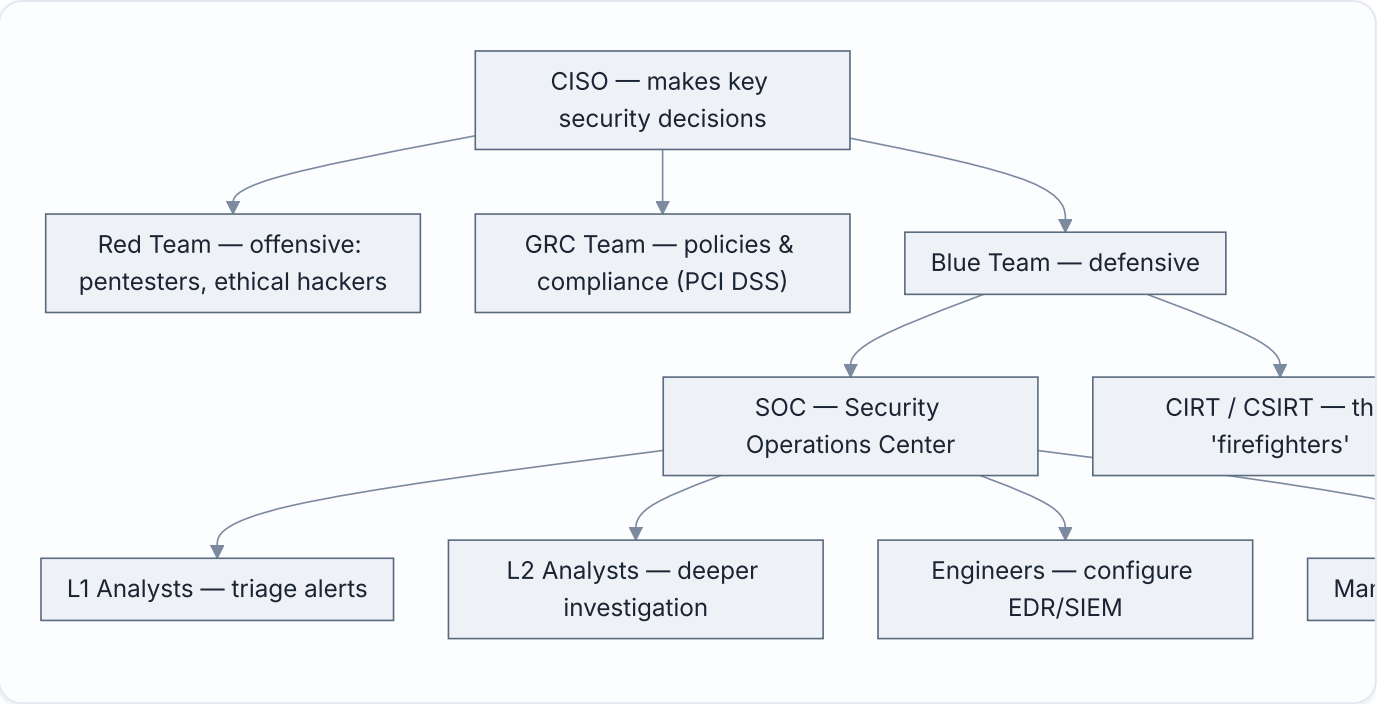
Supply chain

Your apps depend on thousands of libraries. If attackers compromise one and push a malicious update, everyone who installs it is hit — hard to defend because you can't control all upstream software.

13 · The SOC & Blue Team Structure

What is the Blue Team?

The **Blue Team** is defensive security — constantly monitoring for attacks and responding fast. Its size and shape depend on the company, but the **SOC** is where most careers begin.



📘 **SOC**

The **Security Operations Center** — an organisation's defensive hub, often 24/7/365. The first line of defence, it works through **alerts** and handles most attacks. A typical day: review alerts, investigate anomalies, respond to incidents.

SOC roles

Role	Responsibility
L1 Analyst	Triage alerts; escalate complex cases to L2 (you start here)
L2 Analyst	Investigate advanced attacks
Engineer	Configure & maintain security tools (EDR, SIEM)
Manager	Run the SOC, report results upward

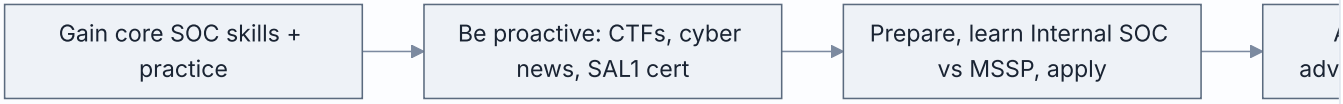
CIRT & specialised roles

When the SOC is overwhelmed, call **CIRT** (a.k.a. CSIRT/CERT) — the firefighters (e.g. JPCERT, Mandiant, AWS CIRT). Larger orgs add specialists: **Digital Forensics Analyst, Threat Intelligence Analyst, AppSec Engineer, AI Researcher**.

The Security Hierarchy

Priorities differ per company: a law firm protects document *confidentiality*; a factory, production *availability*; a hospital, patient *safety*. Top executives (CEO) focus on business; they hire a **CISO** (Chief Information Security Officer) to lead security.

Your SOC Career Path



📘 **INTERNAL SOC VS MSSP**

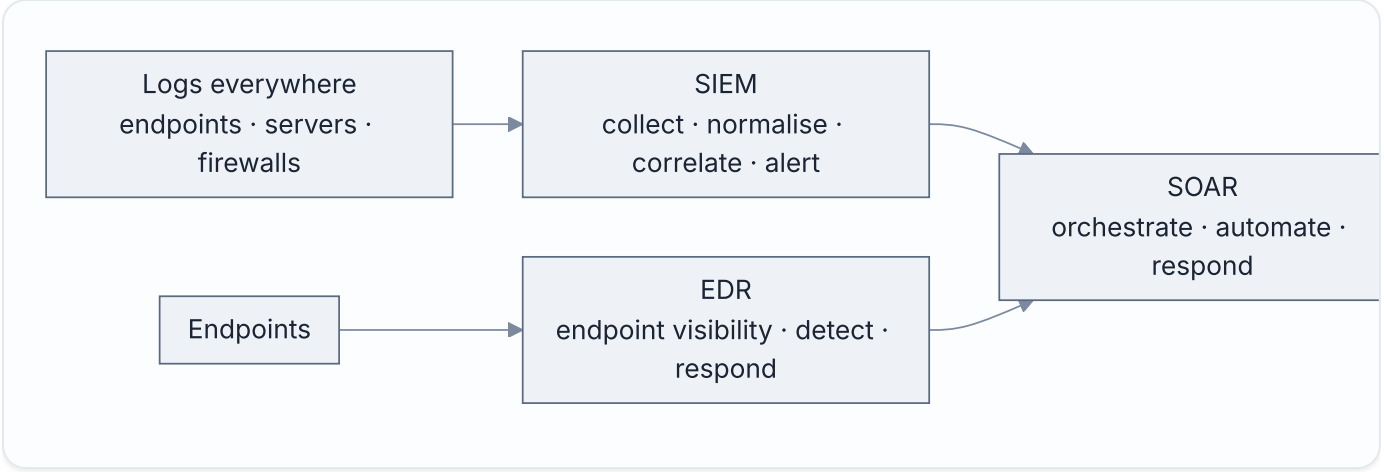
	Internal SOC	**MSSP (Managed Security Service Provider)**
Example	Protect your own company (a bank)	Protect 60 customers for a provider
Pace	Calmer shifts	Often starts with a queue of urgent alerts
Tools	A few, known deeply	Many diverse platforms
Incident practice	Rarer, deeper	Weekly — learn from constant attacks

💡 **FOUR HABITS OF A GOOD ANALYST**

Learn from every alert · think like an attacker · verify everything · never stop learning. Your natural next step after L1 is **L2**, but engineering, IR, or threat intel may call to you instead.

14 · Core SOC Tools: SIEM · EDR · SOAR

Three tool classes define the modern SOC. SIEM centralises and correlates logs; EDR watches the endpoints; SOAR automates the whole workflow.



SIEM — Security Information & Event Management

🔑 **THE PROBLEM SIEM SOLVES**

Networks produce **hundreds of events per second** across scattered devices in different formats. Analysing them individually is impossible: no centralisation, limited context, limited analysis, format chaos.

Log source types

Type	Generated by	Examples
Host-centric	Windows, Linux, servers	File access, authentication, process execution, registry edits, PowerShell
Network-centric	Firewalls, IDS/IPS, routers	SSH, FTP, web traffic, VPN logins, file sharing

What SIEM does

Centralised collection · **Normalisation** (raw logs → consistent fields) · **Correlation** (links events across sources) · **Real-time alerting** · **Dashboards & reporting.**

📘 **CORRELATION EXAMPLE**

Individually harmless events — Haris logs in via VPN from a *new* IP → accesses a shared drive → runs a PowerShell script → system makes an outbound connection — **correlated**, they point to data exfiltration from compromised VPN creds.

Detection rules & alert investigation

```
If LogSource = WinEventLog AND EventID = 104  → alert "Event Log Cleared"
If LogSource = WinEventLog AND EventCode = 4688 AND NewProcessName contains "whoami"
                                              → alert "WHOAMI Execution Detected"
```

After an alert fires, decide: **False Positive** (tune the rule) · **True Positive** (investigate) · contact the asset owner · isolate host · block IP.

Log ingestion methods

Agent/Forwarder (Splunk forwarder) · **Syslog** · **Manual upload** (offline data) · **Port-forwarding** (SIEM listens on a port).

EDR — Endpoint Detection & Response

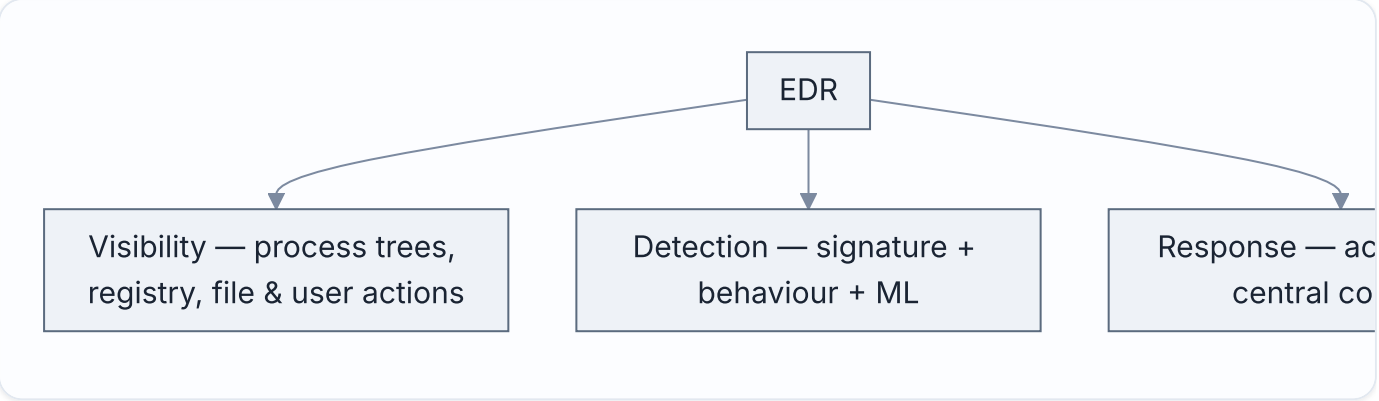
📘 **AV VS EDR — THE AIRPORT ANALOGY**

Antivirus = immigration check: stops *known* criminals (signatures) but waves through an innocent-looking newcomer. **EDR** = security officers *inside* the airport, watching behaviour continuously and acting on anything suspicious. EDR adds behavioural detection over AV's signatures.

AV vs EDR through one attack

Attack step	AV response	EDR response
Phishing Word doc downloaded	Nothing (no signature)	Logs & monitors the download
User opens document	Nothing (winword.exe is legit)	Records execution, keeps watching
Macro spawns PowerShell	Nothing (no signature)	**Flags** the unusual winword→PowerShell parent-child
Obfuscated PowerShell downloads payload	Usually nothing	Flags obfuscated script
Payload injected into `svchost.exe`	Nothing (no memory monitoring)	Detects process injection
Attacker gets remote access	No network visibility	Flags svchost.exe's outbound connection
Final	May mark **clean**	Generates an **alert with the full attack chain**

The three pillars & telemetry



Telemetry collected (the endpoint's "black box"): process executions/terminations · network connections · command-line activity · file/folder modifications · registry modifications.

Detection techniques: Behavioural · Anomaly (learns a baseline) · **IOC matching** · **MITRE ATT&CK mapping** · Machine Learning (catches fileless/multi-stage chains).

Response actions: Isolate Host · **Terminate Process** · **Quarantine** file · **Remote Access** (RTR shell) · **Artefact collection** (memory dump, event logs, registry hives).

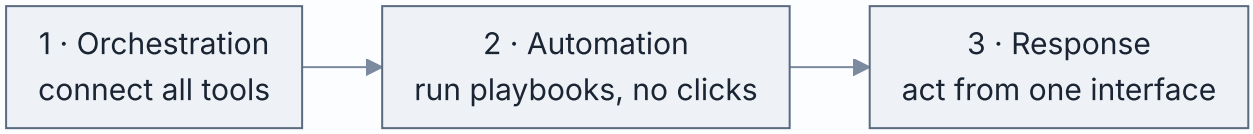
⚠️ **EDR'S LIMIT**

EDR is **host-only** — it does *not* see network-level threats. That's why it works alongside SIEM, NDR, and firewalls.

SOAR — Security Orchestration, Automation & Response

🔑 **WHAT SOAR FIXES**

Traditional SOCs suffer **alert fatigue, too many disconnected tools, manual processes, and talent shortage**. SOAR unifies all tools so analysts don't switch between SIEM, EDR, firewall, TI and ticketing.



A VPN brute-force playbook, automated

1. SIEM sends the alert → 2. query SIEM for the user's historical logins → 3. verify the IP's reputation via Threat Intel → 4. if malicious, disable the user in IAM → 5. open a ticket with full details.

📘 **PLAYBOOK**

A predefined workflow telling SOAR what to do for a category of alert. Playbooks are **dynamic** — each step's result chooses the next path (a phishing playbook, a CVE-patching playbook that fetches CVEs from advisory lists, creates a patch ticket, tests before production, and builds a mitigation plan if assets stay vulnerable).

💡 **DO SOC ANALYSTS STILL MATTER?**

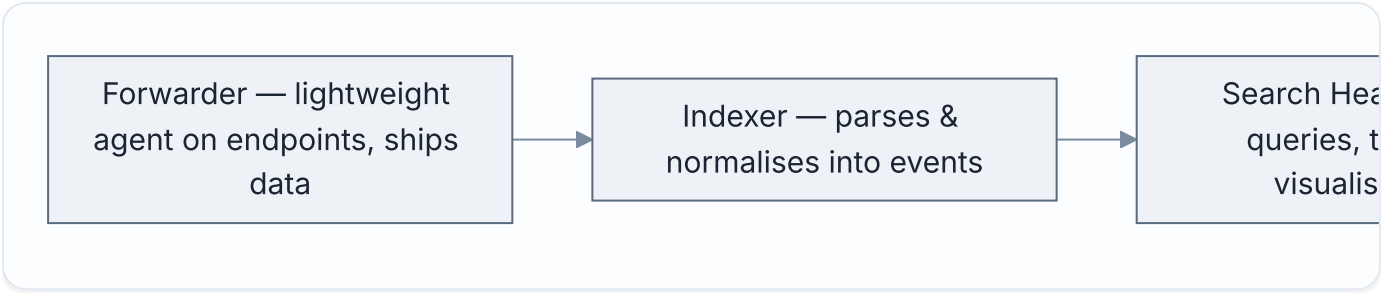
Yes. SOAR automates repetitive tasks but **can't make judgement calls** at critical decision points. Complex investigations still need a human.

Splunk & the Elastic Stack

Two SIEM platforms you'll actually touch.

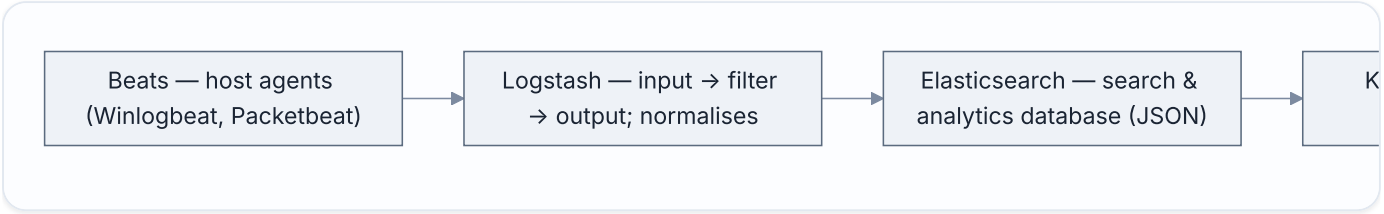
Splunk

Three components:



Ingest data via **Add Data → Upload** (5 steps: select source → source type → input settings/index → review → done). Search with **SPL** (Search Processing Language). Home screen: Splunk Bar, Apps Panel (default *Search & Reporting*), Explore Splunk, Home Dashboard.

Elastic Stack (ELK)



Component	Role
Elasticsearch	Full-text search/analytics engine (stores JSON docs)
Logstash	Ingest → filter/normalise → output
Beats	Single-purpose host data-shippers
Kibana	Web UI for analysis & dashboards

📘 KQL — KIBANA QUERY LANGUAGE

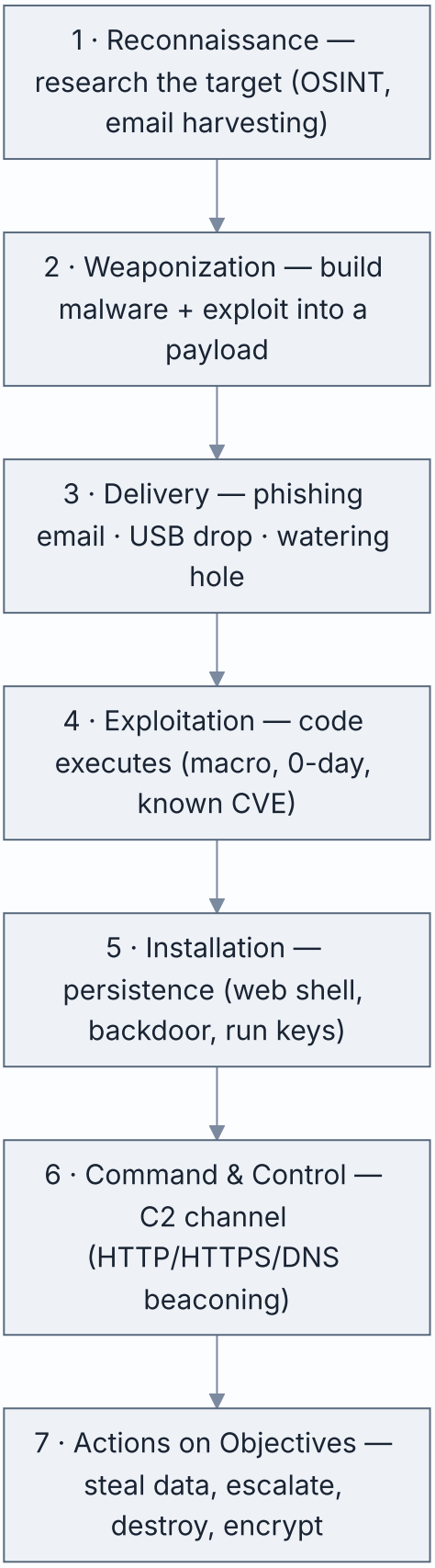
Search in the **Discover tab** two ways: **free-text** (`United*` — `*` wildcard; KQL matches whole terms) and **field-based** (`Source_ip : 238.163.231.224 AND UserName : Suleman`). Logical operators `AND / OR / NOT` . Build **Visualizations** (tables, pie charts) and combine them into **Dashboards**. The Discover tab (logs, fields pane, index pattern, search bar, time filter, timeline) is where analysts spend most time.

15 · Cyber Defense Frameworks

Frameworks give defenders a shared language and a map of how attacks unfold. Master these four.

Cyber Kill Chain

Lockheed Martin's 2011 model (from a military concept) defines the **7 stages** of an intrusion. Breaking any link disrupts the attack.

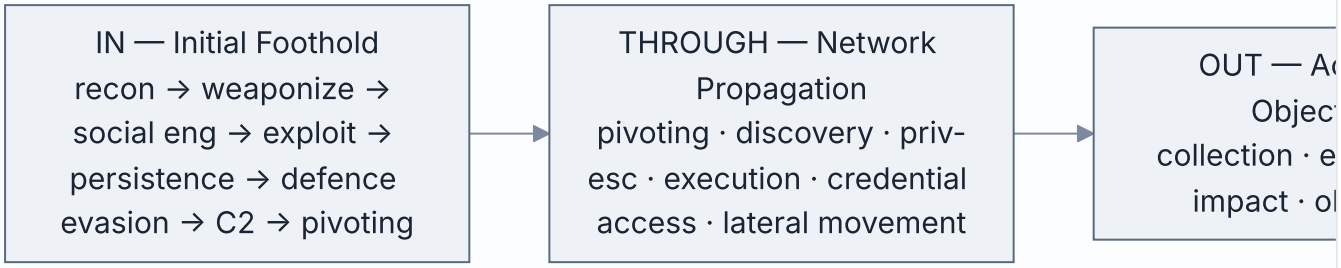

1 · Reconnaissance — research the target (OSINT, email harvesting)
2 · Weaponization — build malware + exploit into a payload
3 · Delivery — phishing email · USB drop · watering hole
4 · Exploitation — code executes (macro, 0-day, known CVE)
5 · Installation — persistence (web shell, backdoor, run keys)
6 · Command & Control — C2 channel (HTTP/HTTPS/DNS beaconing)
7 · Actions on Objectives — steal data, escalate, destroy, encrypt

📘 **KEY TERMS**

Malware = software designed to damage/gain access · **Exploit** = code abusing a vulnerability · **Payload** = the malicious code that runs. **Passive recon** = no direct contact (WHOIS, social media); **Active recon** = direct (port scanning, social engineering). Email-harvesting tools: theHarvester, Hunter.io, OSINT Framework.

Unified Kill Chain (UKC)

Paul Pols' 2017 framework **complements** (not competes with) the Cyber Kill Chain, with **18 phases** grouped into three goals — and crucially, it accepts attackers loop *back and forth*.



UKC benefit	vs other frameworks
Modern (2017, updated 2022)	MITRE's base is 2013
Very detailed (18 phases)	Others have a handful
Covers the *whole* attack end-to-end	Others cover limited phases
Models realistic back-and-forth	Others assume linear flow

📘 **THREAT MODELLING**

Steps to improve a system's security: (1) identify systems/functions to secure, (2) assess vulnerabilities & how they'd be exploited, (3) plan to secure them, (4) add policies (e.g. an SDLC) to prevent recurrence. Frameworks: **STRIDE, DREAD, CVSS**. The UKC helps identify attack surfaces for modelling.

MITRE

MITRE is a non-profit maintaining several essential knowledge bases.

ATT&CK Framework

> 🔑 **TACTIC · TECHNIQUE · PROCEDURE**
>
> - **Tactic** = the adversary's *goal* (the **why**) — e.g. Reconnaissance.
> - **Technique** = *how* they achieve it — e.g. Active Scanning.
> - **Sub-technique** = a specific method — e.g. Scanning IP Blocks.
> - **Procedure** = the actual implementation.

A globally-accessible knowledge base of adversary **TTPs** from real-world observations, covering Windows, macOS, Linux, cloud and more. The **ATT&CK Matrix** / **Navigator** visualise it.

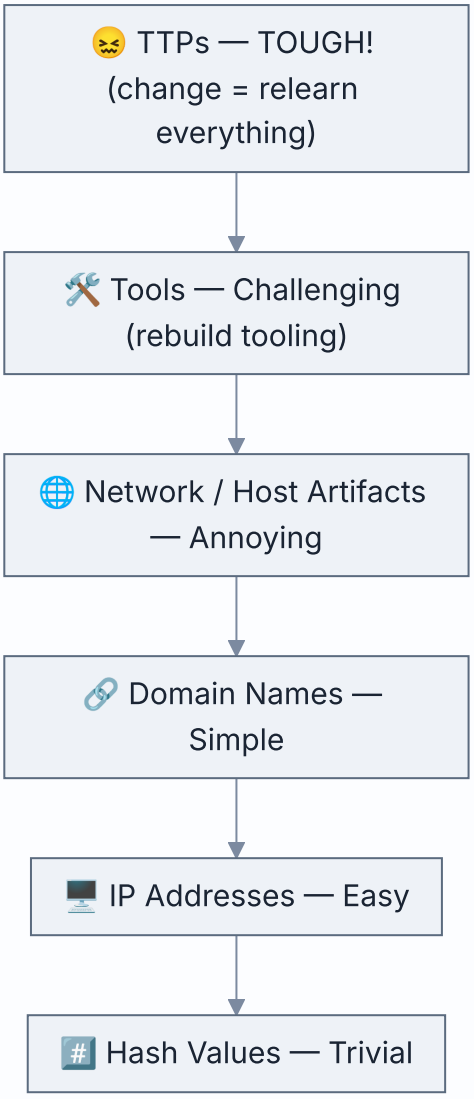
Who uses ATT&CK	How
CTI teams	Map threat-actor behaviour to TTPs → actionable profiles
SOC Analysts	Add context to alerts, prioritise incidents
Detection Engineers	Map SIEM/EDR rules to ATT&CK for coverage
Incident Responders	Map timelines to tactics/techniques
Red & Purple Teams	Build emulation plans

MITRE's other projects

Project	Purpose
CAR (Cyber Analytics Repository)	Ready-made detection analytics with pseudocode + Splunk/LogPoint queries (e.g. CAR-2020-09-001)
D3FEND	The *defensive* counterpart — maps defensive techniques (Model, Harden, Detect, Isolate, Deceive, Evict); e.g. Credential Rotation D3-CRO
Emulation Library / Caldera	Adversary emulation plans & automated emulation tool
AADAPT / ATLAS	Newer matrices for digital-asset payments / AI systems threats

The Pyramid of Pain

David Bianco's model ranks indicators by **how much pain** detecting them causes the attacker. The higher you detect, the harder (and more expensive) it is for them to adapt.



Indicator	Pain	Why
Hash values	Trivial	Change one bit → totally new hash (`echo x >> file`)
IP addresses	Easy	Attacker rotates IPs; *Fast Flux* evades blocking
Domain names	Simple	Must buy/register/change DNS; watch for Punycode & URL shorteners
Host artifacts	Annoying	Registry values, dropped files, suspicious process spawns
Network artifacts	Annoying	User-Agent strings, C2 URIs, HTTP POST patterns (detect in PCAPs with TShark)
Tools	Challenging	Detecting tools forces attacker to rebuild — use AV sigs, detection rules, **YARA**, fuzzy hashing (SSDeep)
TTPs	Tough!	Detecting *behaviour* (e.g. Pass-the-Hash) leaves them to retrain or give up

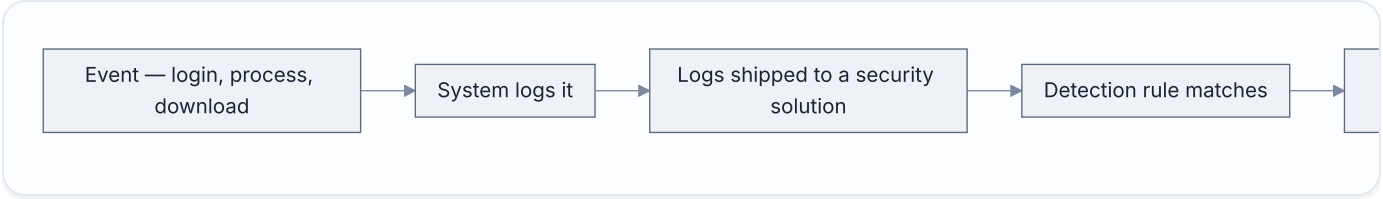
🔑 **BIANCO'S PRINCIPLE**

"The amount of pain you cause an adversary depends on the types of indicators you are able to make use of." Climbing the pyramid shifts you from blocking throwaway indicators to disrupting the attacker's very playbook. **Hashing algorithms:** MD5, SHA-1, SHA-2 (look up on VirusTotal / MetaDefender).

16 · Alert Triage, Reporting & Metrics

The day-to-day craft of an L1 analyst: turning a flood of alerts into decisions, documenting them, and measuring the team's effectiveness.

From Events to Alerts



Alert management platforms: SIEM (Splunk ES, Elastic) · EDR/NDR (MS Defender, CrowdStrike) · SOAR (Splunk/Cortex SOAR) · ITSM (Jira, TheHive).

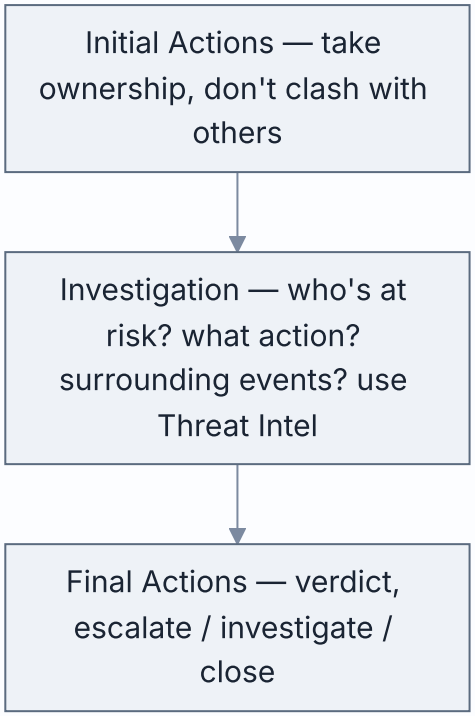
Alert Properties

#	Property	Meaning
1	**Alert Time**	When the alert was created (minutes after the event)
2	**Alert Name**	Summary from the rule (e.g. *Unusual Login Location*)
3	**Severity**	🟢 Low · 🟡 Medium · 🟠 High · 🔴 Critical
4	**Status**	🆕 New · 🔄 In Progress · ✅ Closed
5	**Verdict**	🔴 True Positive · 🟢 False Positive
6	**Assignee**	The owner responsible
7	**Description**	Rule logic, why it's suspicious, how to triage
8	**Fields**	Values that triggered it (hostname, command line...)

Alert Prioritisation & Triage

🔑 **PRIORITISATION ORDER**

1. **Filter** — only take new, unreviewed alerts. 2. **Sort by severity** — critical → high → medium → low. 3. **Sort by time** — oldest first (older breaches are further along).



📘 **FOUR TRIAGE FACTORS (ADVENT OF CYBER)**

Factor	Ask	Why
Severity	How bad?	Urgency & business risk
Time	When?	Spot ongoing / repeated attacks
Context	Where in the attack lifecycle?	How far has the attacker progressed?
Impact	Who/what is affected?	Prioritise by asset importance

Investigation steps: investigate the alert in detail → check related logs → **correlate** multiple alerts (same user/IP/device reveals the attack path) → build a timeline → decide the next action → **document findings**.

Reporting & Escalation

🔑 **WHY L1 ANALYSTS WRITE REPORTS**

- **Context for escalation** — saves L2 huge time.
- **Records** — raw SIEM logs last 3–12 months, but *alerts are kept indefinitely*.
- **Skill growth** — *"if you can't explain it simply, you don't understand it."*

Report format — the Five Ws

W	Capture
Who	Which user logged in / ran the command / downloaded the file
What	The exact action or event sequence
When	Start & end of the suspicious activity
Where	Device, IP, or website involved
Why	*(most important)* your reasoning for the verdict

When to escalate to L2

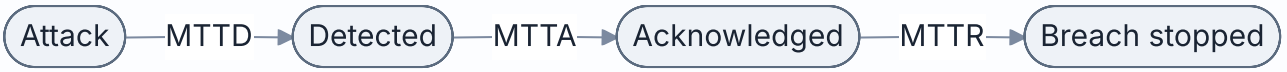
1. Indicator of a major cyberattack needing deeper investigation/DFIR.
2. Remediation needed (malware removal, host isolation, password reset).
3. Communication with customers/management/law enforcement required.
4. You simply don't fully understand it — asking is better than blindly closing.

Steps: write report + verdict → move alert to *In Progress* → reassign to L2 on shift → ping them.

⚠️ **COMMUNICATION EDGE CASES**

L2 unresponsive 30 min on a critical alert → call L2, then L3, then manager. Slack/Teams account compromise → verify the user via **another channel** (phone), never the breached chat. Realise you misclassified days later → tell L2 *immediately* (actors stay silent for weeks).

SOC Metrics & Objectives



Metric	Formula	Measures	Common SLA
Alerts Count	total alerts received	Analyst workload	—
False Positive Rate	FP / total alerts	Noise level	—
Alert Escalation Rate	escalated / total	L1 experience	—
Threat Detection Rate	detected / total threats	SOC reliability	—
MTTD	—	Attack → detection	5 min
MTTA	—	Alert → triage starts	10 min
MTTR	—	Alert → breach stopped	60 min

💡 **WORKED EXAMPLE**

Alert after **12 min**, moved to In Progress **10 min** later, escalated after **6 min**, L2 spent **35 min** cleaning → **MTTD 12, MTTA 10, MTTR 51** (12+6+… = 10 to ack; 12+6+35−… the room's answer: **12, 10, 51**). A 94 % false-positive rate means analysts grow numb and miss real threats — tune rules.

SOC Workbooks & Lookups

📘 **WORKBOOK / PLAYBOOK / RUNBOOK**

A structured document defining the steps to investigate & remediate a specific threat consistently — vital because L1 analysts are junior. Example: an *Unusual Login Location* workbook (enrichment → investigation → escalation).

To triage "G.Baker logged into HQ-FINFS-02 and shared a file with R.Lund," you need context from:

Lookup	Holds	Sources
Identity Inventory	Users/services: role, contacts, privileges	AD/Entra ID, SSO (Okta), HR systems, CSV/Excel
Asset Inventory	Hosts: location, IP, OS, owner, purpose	AD, SIEM/EDR, MDM (Intune/Jamf), CSV/Excel
Network Diagrams	Subnets, firewall rules, services	Documentation

A network diagram turns raw IPs into a story: an external IP brute-forces VPN (port 10443) → gets a VPN-subnet IP → scans the Database subnet (blocked by firewall) → pivots to the Office subnet.

17 · Offensive Web & AI Labs

Hands-on web-application attacks from Advent of Cyber 2025. Each builds on How the Web Works.

AI in Cyber Security

AI agents now assist across the whole field:

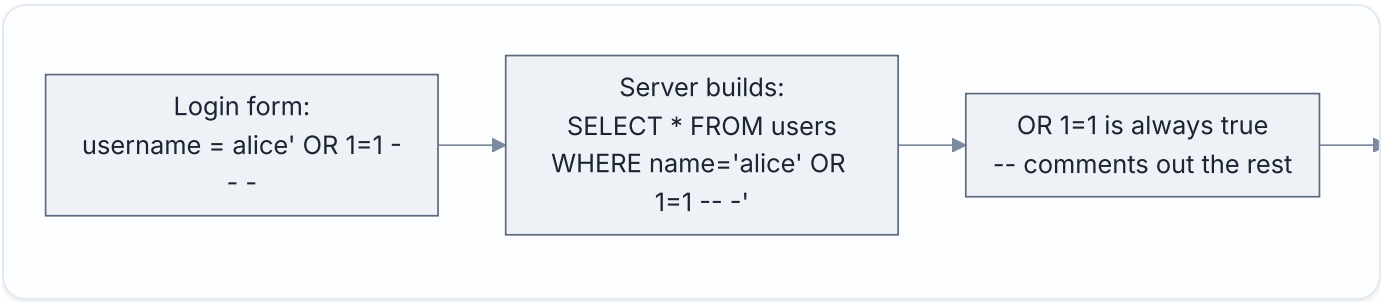
Area	AI's role
Red (offensive)	Automate recon/OSINT, analyse noisy scanner output, generate exploit scripts
Blue (defensive)	Process telemetry continuously; speed detection, investigation & automated response (isolate device, block email)
Software	A "virtual colleague"; SAST/DAST vuln-finding — *ironically better at finding bugs than writing secure code*

> ⚠️ **CONSIDERATIONS**
>
> You don't *own* AI output; decisions may be opaque or biased; it can be wrong. Never assume 100 % accuracy — especially in offensive testing where a mistake hits a real client.

The lab generated a **SQL-injection** exploit (`username = "alice' OR 1=1 -- -"`), analysed the resulting web log, and reviewed vulnerable PHP. Fix SQLi with **prepared/parameterised queries** and input sanitisation.

SQL Injection & Secure Coding

> 📘 **SQL INJECTION (SQLI)**
>
> When unsanitised user input is concatenated into a database query, an attacker can **change the query's logic** — bypassing login, reading other tables, or dumping the database. See Databases & SQL for the query basics.



Payload fragment	Effect
`' OR 1=1 -- -`	Makes the `WHERE` always true; `--` comments out the rest
`' UNION SELECT ...`	Appends attacker-chosen columns to the result
`'; DROP TABLE ...`	Stacks a second, destructive statement

> ⚠️ **WHY THE VULNERABLE PHP FAILED**
>
> `$user = $_POST['username'] ?? ''` takes raw input and drops it straight into the query — no validation, no escaping. Any quote the attacker types becomes part of the SQL.

Secure-coding fixes (defence in depth):

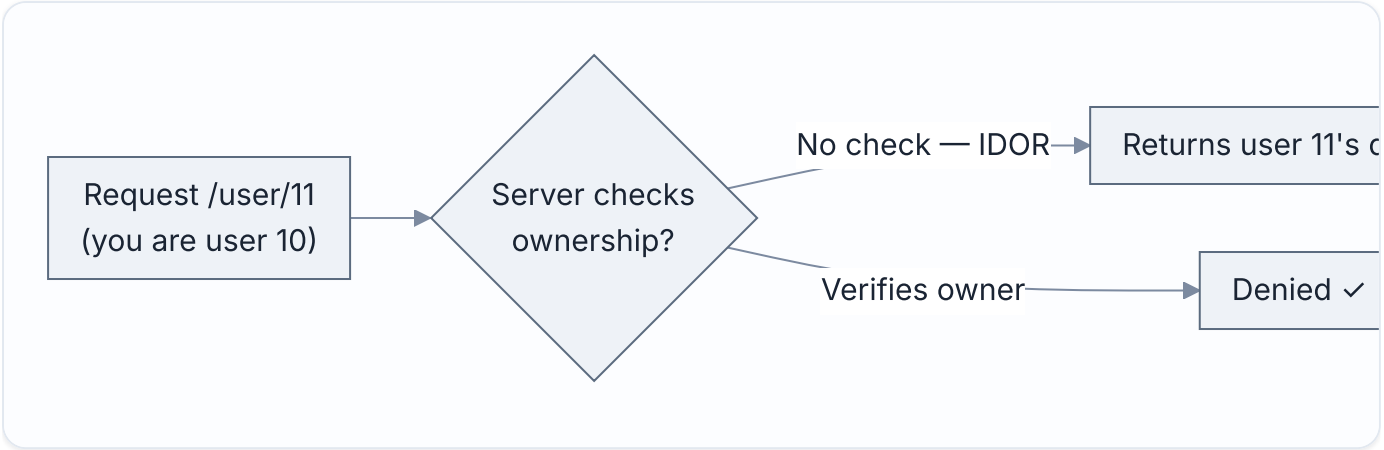
Control	What it does
Prepared / parameterised queries	Data is bound separately from SQL — input can *never* become code (the single most important fix)
Input validation	Reject anything not matching the expected format
Sanitisation / escaping	`filter_input`, `htmlspecialchars` for defence in depth
Least-privilege DB account	The web user can't `DROP` tables even if injected
WAF	Blocks common injection patterns at the edge

> 💡 **TEST IT YOURSELF**
>
> Tools like **OWASP ZAP** (scanning) and **SQLmap** (automated injection testing) confirm whether an endpoint is vulnerable — only against systems you're authorised to test.

IDOR — Insecure Direct Object Reference

📘 **IDOR (A.K.A. AUTHORIZATION BYPASS)**

An access-control flaw: the app uses a reference (like `/user/1`) to fetch data but **never checks you're allowed** that object. Change the ID → see someone else's data.



🔑 **AUTHENTICATION VS AUTHORIZATION**

Authentication = verifying *who you are* (username + password → a cookie/token). **Authorization** = verifying *what you're allowed*. Authorization can't happen before authentication. IDOR is usually **horizontal privilege escalation** (same privilege level, other users' data) vs **vertical** (gaining more features).

IDOR disguises (still exploitable): plain numbers → encoding (`Mg==` = Base64 of `2`) → hashes (MD5/SHA1 of a known value) → **UUIDv1** (recoverable if you know the generation timestamp).

💡 **THE FIX**

Check ownership **server-side on every request** — don't rely on hiding/encoding IDs. Use hard-to-guess IDs for public links *as well*, and log/monitor failed access attempts.

Cross-Site Scripting (XSS)

Injecting malicious JavaScript into input fields that get reflected to other users.

Type	Where the script lives	Victim
Reflected	In the request/URL, echoed immediately	Whoever clicks the crafted link (via phishing)
Stored	Saved on the server (e.g. a comment)	**Every** user who views the page — "set and forget"

```
<!-- test payload -->
<script>alert('Meow Meow')</script>
```

> ⚠️ **IMPACT**
>
> Steal session cookies, trigger fake login popups, deface pages, or act as the victim.

Defences: use `textContent` not `innerHTML` · set cookies `HttpOnly` + `Secure` · **sanitise & encode** input/output · if HTML must be allowed, filter strictly.

Race Conditions

> 📘 **RACE CONDITION**
>
> When two+ actions happen simultaneously and the outcome depends on finishing order. In web apps, concurrent requests can manipulate stock or balances.

Type	What happens
TOCTOU (Time-of-Check to Time-of-Use)	Data changes between the check and the use
Shared resource	Multiple writes to the same data without control
Atomicity violation	A "do it all at once" operation runs in separable parts

The exploit (Burp Suite): make one legitimate checkout → capture the `POST /process_checkout` → send to Repeater → duplicate the tab ~15× into a group → **Send group in parallel (last-byte sync)** → stock goes negative, multiple orders confirm.

> 💡 **MITIGATION**
>
> **Atomic DB transactions** · final stock validation before commit · **idempotency keys** · rate limiting / concurrency controls.

Exploitation with cURL

`curl` speaks HTTP from the command line — ideal when GUI tools aren't available.

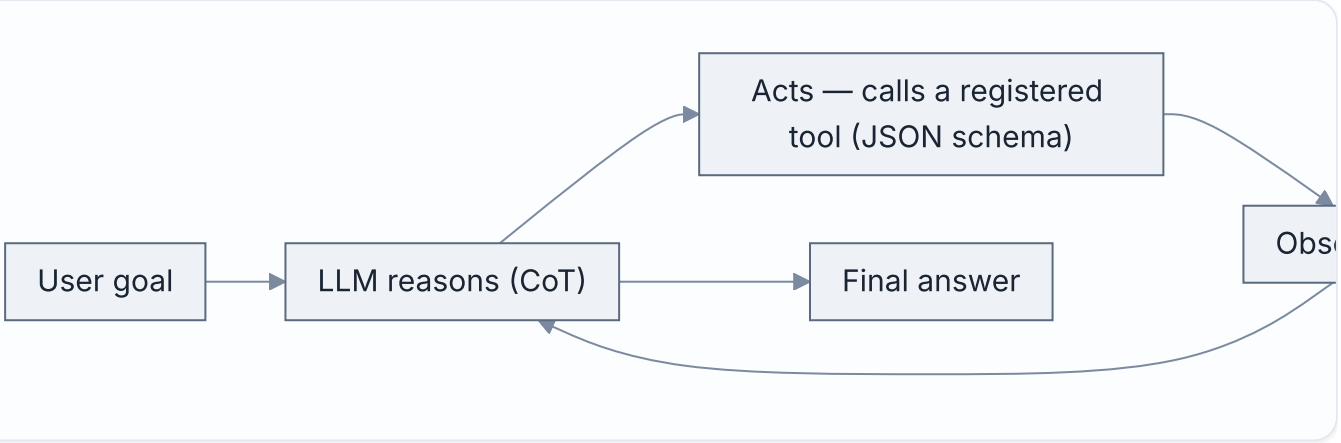
```
curl http://MACHINE_IP/                                    # GET
curl -X POST -d "username=user&password=user" .../post.php   # POST
curl -i -X POST -d "user=admin&pass=admin" .../post.php      # -i shows headers
curl -c cookies.txt -d "username=admin&password=admin" .../session.php  # save cookies
curl -b cookies.txt .../session.php                        # reuse cookies
curl -A "internalcomputer" .../ua_check.php                # spoof User-Agent
```

Brute-force loop (the logic behind Hydra / Burp Intruder / WFuzz):

```
for pass in $(cat passwords.txt); do
  response=$(curl -s -X POST -d "username=admin&password=$pass" http://MACHINE_IP/brutef
  echo "$response" | grep -q "Welcome" && { echo "[+] Found: $pass"; break; }
done
```

Agentic AI & Prompt Injection

> 📘 **AGENTIC AI**
>
> AI with *agency* — it can **plan** multi-step tasks, **act** (run tools, call APIs, copy files), and **watch & adapt**. Powered by **LLMs** using **Chain-of-Thought** reasoning and the **ReAct** (Reason + Act) pattern — alternating reasoning traces with actions.



⚠️ **THE RISK**

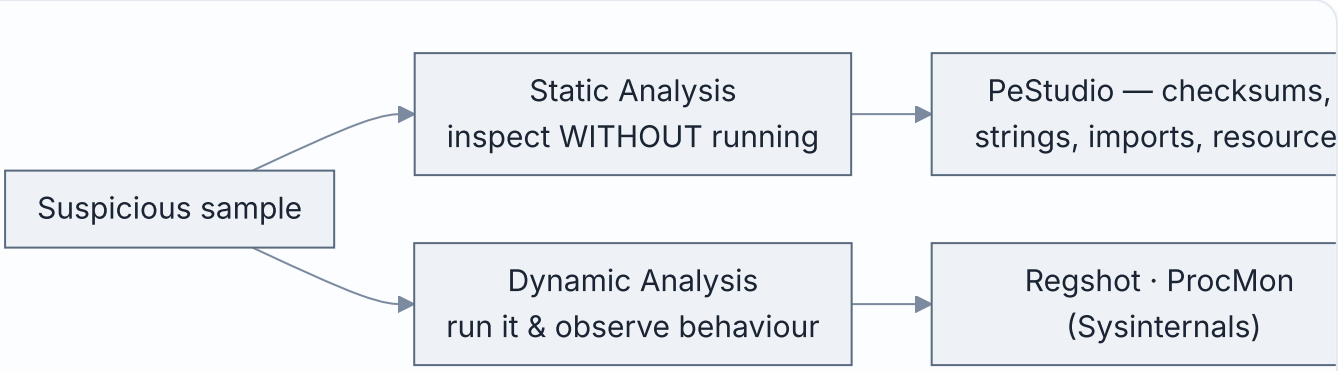
Tools are registered to the model via JSON schemas (e.g. `web_search(query)`). When an agent follows a process to complete tasks, attackers can **interfere with that process** through crafted input — **prompt injection** (plus jailbreaking and data poisoning) — forcing unsafe or unintended actions. Strong input handling and tool-permission controls are essential.

18 · Malware, Forensics & Detection Labs

The defensive hands-on core: analysing malware, decoding obfuscation, hunting in logs and the registry, and detecting C2.

Malware Analysis Fundamentals

> 🔑 **THE GOLDEN RULE**
>
> **Never run dangerous applications on devices you care about.** Analyse samples inside a safe, isolated **sandbox** (usually a VM with snapshotting). See Virtualization.



Static analysis — what PeStudio reveals

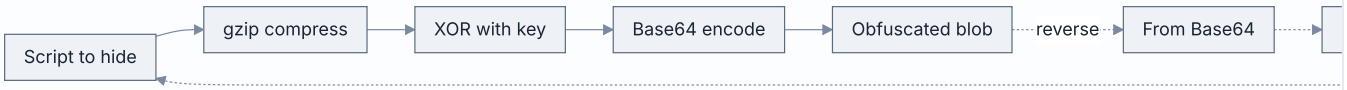
Information	Why it matters
Checksums	Google the hash to see if it's known malware
Strings	Readable text: IPs, URLs, commands, even passwords
Imports	Libraries/functions it uses (e.g. `CreateFileW`) → how it interacts with the OS
Resources	Icons & embedded data — malware hides here (fake Word icon)

Dynamic analysis tools

- **Regshot** — snapshots the registry before/after to spot changes (malware adds a **Run key** for persistence).
- **ProcMon** — monitors real-time process/registry/file/network interactions.

Obfuscation & CyberChef

> 📘 **ENCODING VS ENCRYPTION VS OBFUSCATION**
>
> **Encoding** transforms data for compatibility (reversible, no key). **Encryption** protects confidentiality (needs a key). **Obfuscation** makes data hard to read to evade detection & delay analysis.



Technique	Visual clue
ROT1 / ROT13	Words look "one/13 letters off"; spaces unchanged (`the` → `gur`)
Base64	Long alphanumeric strings, `+` `/` , ending `=` / `==`
XOR	Random symbols, *same length* as original; short reused keys repeat

CyberChef — the Cyber Swiss Army Knife

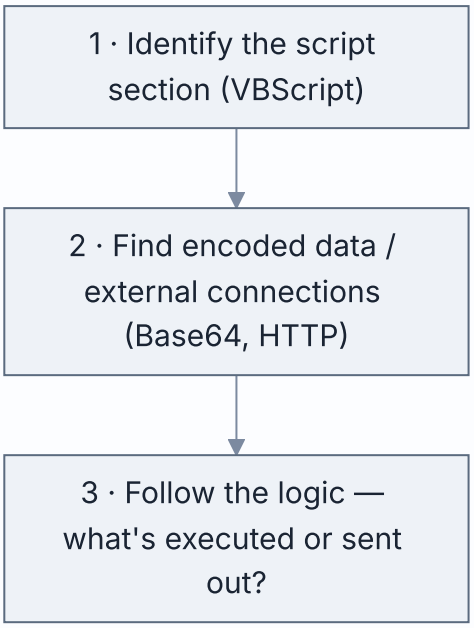
Four areas: **Operations** (capabilities) → drag into **Recipe** (chain them) → **Input** → **Output**. Apply layered obfuscation in **reverse order** to recover plaintext. The **Magic** operation auto-guesses common decoders (won't catch custom XOR keys).

> 💡 **XOR'S MAGIC PROPERTY**
>
> XOR-ing the result with the **same key again** returns the original data. Two XOR operations with one key = identity. Used to reverse XOR-encoded payloads (and in the CyberChef "lock" challenges, chained with Base64 and hashing → CrackStation for MD5).

HTA Malware Analysis

> 📘 **HTA FILES**
>
> **HTML Application** files run via `mshta.exe` — HTML + CSS + VBScript/JavaScript as a desktop app. Legitimately used for admin tools; abused for **initial access, downloaders/droppers, obfuscation, and living-off-the-land** (calling `mshta.exe`, `powershell.exe`, `wscript.exe`, `rundll32.exe`).

Three-step analysis process:



Check `<title>` & `HTA:APPLICATION` first (fake names like "Salary Survey"). Watch `CreateObject()` calls: **InternetExplorer.Application** (external connection), **WScript.Network** (recon), **WScript.Shell** (run commands). Trace variable chains — in the sample, `$U` (decoded URL) → `$C` (downloaded content) → `$B` (runs it as a scriptblock in memory).

Web Attack Forensics (Splunk)

Reconstruct an attack chain across log sources with SPL.

Query goal	SPL
Suspicious web commands	`index=windows_apache_access (cmd.exe OR powershell OR "Invoke-Expression")`
Server-side errors/exec	`index=windows_apache_error ("cmd.exe" OR "powershell" OR "Internal Server Error")`
Apache spawning processes	`index=windows_sysmon ParentImage="*httpd.exe"`
Enumeration confirmed	`index=windows_sysmon *cmd.exe* *whoami*`
Encoded PowerShell	`index=windows_sysmon Image="*powershell.exe" (CommandLine="*enc*" OR "*-EncodedCommand*")`

🔑 **THE SMOKING GUN**

If `httpd.exe` **(Apache) spawns** `cmd.exe` **or** `powershell.exe`, that's a **command injection** that reached the OS — Apache should only spawn worker threads. Decode Base64 PowerShell to see the attacker's true intent.

Windows Registry Forensics

📘 **THE WINDOWS REGISTRY**

The OS's "brain" — a database of all configuration, stored across several **hives** (files).

Hive	Contains	In Registry Editor
SYSTEM	Services, drivers, boot, mounted devices	`HKLM\SYSTEM`
SECURITY	Local security & audit policy	`HKLM\SECURITY`
SOFTWARE	Installed programs, OS info, autostarts	`HKLM\SOFTWARE`
SAM	Usernames, password hashes, groups	`HKLM\SAM`
NTUSER.DAT	Recent files, user prefs, user autostarts	`HKU\<SID>` / `HKCU`
USRCLASS.DAT	Shellbags, jump lists	`HKU\<SID>\Software\Classes`

Forensically useful keys: **USBSTOR** (connected USBs) · **RunMRU** (Run-dialog commands) · **UserAssist** (GUI-launched apps) · **TypedPaths** / **WordWheelQuery** (Explorer address bar / search)

· **Run** (startup/persistence) · **RecentDocs** (recent files). Use **Registry Explorer** (Eric Zimmerman) for deep analysis — the built-in editor only shows live data.

C2 Detection with RITA

> 📘 **RITA + ZEEK**
>
> **RITA** (Real Intelligence Threat Analytics) detects **C2 communication** by analysing network logs. It only accepts **Zeek** logs (Zeek is a network-security-monitoring tool, not an IDS). Convert a PCAP first: `zeek readpcap capture.pcap output_dir/`.

```
zeek readpcap pcaps/AsyncRAT.pcap zeek_logs/asyncrat   # PCAP → Zeek logs
rita import --logs ~/zeek_logs/asyncrat/ --database asyncrat
rita view asyncrat                                      # search bar: /  help: ?
```

RITA detects: **C2 beaconing · DNS tunnelling · long connections · data exfiltration · threat-intel matches**, scoring connections by severity.

Threat modifier	Flags
MIME/URI mismatch	Header MIME type ≠ URI
Rare signature	Unusual user-agent/SSL cert
Prevalence	Few internal hosts talk to one external host
First Seen	New external host = more suspicious
Missing host header	Attacker oversight
Large outgoing data	Possible exfiltration

> 💡 **READING RESULTS**
>
> Long FQDNs (`sunshine-bizrate-inc-software[.]trycloudflare[.]com`), non-standard ports, long durations, and high connection counts all point to C2. Pivot into the Zeek logs/PCAP and confirm malicious IPs on VirusTotal.

YARA Rules

> 📘 **YARA**
>
> A tool to identify & classify malware by matching **patterns** — a detective's notebook for defenders. Used for post-incident analysis, threat hunting, intel-based scans, and memory analysis.

Every rule has three parts:

```
rule TBFC_KingMalhare_Trace {
    meta:                                   // author, description, date
        author = "Defender of SOC-mas"
    strings:                                // the clues to search for
        $s1 = "rundll32.exe" fullword ascii
        $mz = { 4D 5A 90 00 }               // hex: MZ (Windows exe header)
    condition:                              // when to trigger
        $s1 and $mz
}
```

Element	Role
Meta	Info about the rule (optional but recommended)
Strings	Text, hex, or regex clues
Conditions	Logic that fires the rule

String modifiers: `nocase` (ignore case) · `wide ascii` (2-byte + 1-byte) · `xor` (all single-byte XOR variants) · `base64`. **Conditions:** `$xmas` · `any of them` · `all of them` · `($s1 or $s2) and not $benign` · `any of them and (filesize < 700KB)`.

```
yara -r icedid_starter.yar C:\    # -r recursive   -s show matched strings
```

19 · Infrastructure, Cloud & Tooling Labs

Network discovery, password attacks, containers, cloud, industrial systems, and phishing — the remaining Advent of Cyber hands-on labs.

Network Discovery with Nmap

> 🔑 **THE ATTACK FLOW**
>
> 1. Know your target (IP). 2. Scan for open ports (22 SSH, 80 HTTP). 3. Explore the services behind them. 4. Exploit a way in.

```
nmap MACHINE_IP                 # top 1000 ports
nmap -p- MACHINE_IP             # all 65535 TCP ports
nmap -sU MACHINE_IP             # UDP ports (e.g. 53 DNS)
nmap -sV -p 22,80,502 IP        # service/version detection on specific ports
ss -tunlp                       # on-host: list listening ports (TCP/UDP/numeric/process
```

Protocol	Default port	Note
SSH	22	Remote shell
HTTP	80	Web
FTP	21 (or custom, e.g. 21212)	Try `anonymous` login
DNS	53/udp	Hides data in records (`dig @IP TXT name +short`)
MySQL	3306	May allow unauthenticated localhost access

> 💡 **ON-HOST DISCOVERY**
>
> Once you have a shell, don't scan — just ask the OS its listening ports with `ss -tunlp` (or `netstat`). With root you also see the owning process.

Password Attacks

Encryption's strength depends almost entirely on the **password**. Attackers don't break the crypto — they **guess the password offline**.

Attack	How
Dictionary	Try a wordlist (`rockyou.txt` , `common-passwords.txt`)
Mask	Limit to a known format: `?l?l?l?d?d` = 3 lowercase + 2 digits
Brute-force	All combinations (slow; GPU-accelerated with hashcat)

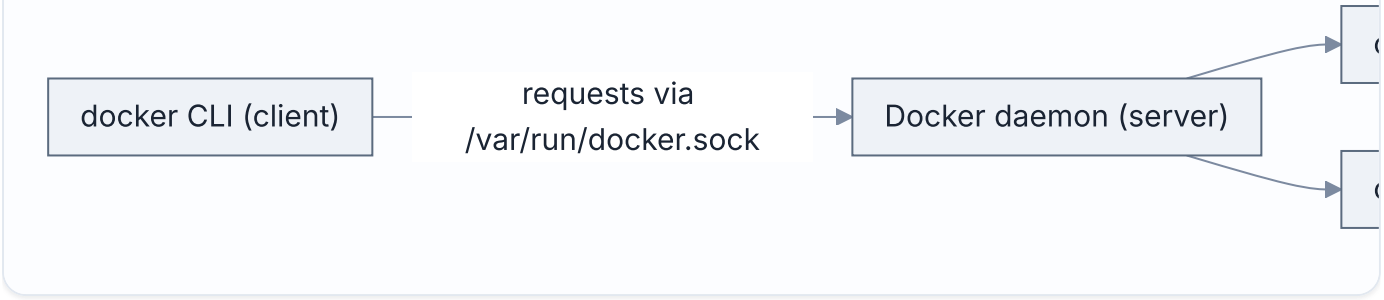
```
file flag.pdf                       # confirm file type first
pdfcrack -f flag.pdf -w rockyou.txt # crack a PDF
zip2john flag.zip > ziphash.txt     # extract a crackable hash
john --wordlist=rockyou.txt ziphash.txt
```

> 📘 **DETECTING CRACKING ON AN ENDPOINT**
>
> Offline cracking doesn't hit login services, so watch the **host**: process names (`john` , `hashcat` , `fcrackzip` , `pdfcrack` , `zip2john`) · command flags (`--wordlist` , `-a 3` , `-m`) · potfiles (`~/.john/john.pot`) · **GPU spikes** (`nvidia-smi` , `nvcuda.dll` / `libcuda.so`) · downloads of `rockyou.txt` /SecLists. Detect with **Sysmon + Sigma** rules.

Response playbook: isolate host → capture triage artefacts → preserve working dir/wordlists/history → review decrypted files → identify origin/intent → remediate & rotate keys/passwords, enforce MFA → educate.

Containers & Docker

See Virtualization for the fundamentals. Docker builds/runs/manages containers sharing the host kernel.



> ⚠️ **CONTAINER ESCAPE**
>
> A **container escape** lets code inside a container reach the host or other containers. If the **Docker socket** (`/var/run/docker.sock`) is mounted into a container, you can run Docker commands *from inside it* (`docker ps`) and reach a **privileged** container — then execute on the host. Mitigation: *Enhanced Container Isolation* blocks socket mounting.

```
docker ps                              # list running containers
docker exec -it uptime-checker sh      # shell into a container
ls -la /var/run/docker.sock            # check socket access
docker exec -it deployer bash          # pivot to a privileged container
```

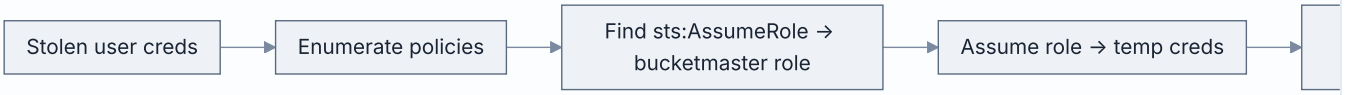
AWS Security

> 📘 **IAM — IDENTITY & ACCESS MANAGEMENT**
>
> AWS's service for managing identities and their permissions.

IAM concept	Meaning
User	A single identity with credentials (password / access keys)
Group	Many users bundled for easier access management
Role	A *temporary* identity a user/service can **assume** for certain permissions
Policy	JSON defining **Action** (allowed) · **Resource** · **Condition** · **Principal** (for whom)

Attacker enumeration flow (credentials in `~/.aws/credentials`):

```
aws sts get-caller-identity                                  # who am I?
aws iam list-users
aws iam list-user-policies --user-name sir.carrotbane    # inline policies
aws iam get-user-policy --policy-name X --user-name ...   # → found sts:AssumeRole
aws iam list-roles                                           # → found "bucketmaster"
aws iam get-role-policy --role-name bucketmaster --policy-name BucketMasterPolicy
aws sts assume-role --role-arn arn:...:role/bucketmaster --role-session-name TBFC
# export the temp AccessKeyId / SecretAccessKey / SessionToken, then:
aws s3api list-buckets
aws s3api get-object --bucket easter-secrets-123145 --key cloud_password.txt out.txt
```



📘 **INLINE VS ATTACHED POLICIES · S3**

Inline policies are hard-coded into one identity (deleted with it); **attached** policies are reusable. **S3** = Simple Storage Service, object storage for any file type.

Industrial Control Systems & Modbus

📘 **SCADA & PLC**

SCADA (Supervisory Control And Data Acquisition) is the command centre of industrial operations — sensors/actuators, **PLCs** (Programmable Logic Controllers — rugged industrial computers running control logic), monitoring systems, and **historians** (data stores).

Modbus (1979) is the industrial request/response protocol — **no authentication, no encryption, no authorisation**. Modbus TCP listens on **port 502**.

Modbus data type	I/O	Values
Coils	Digital output (writable)	0/1
Discrete Inputs	Digital input (read-only)	0/1
Holding Registers	Analogue output (writable)	0–65535
Input Registers	Analogue input (read-only)	0–65535

⚠️ **THE ATTACK & SAFE REMEDIATION**

King Malhare connected to port 502 (no auth) and rewrote registers/coils: HR0=1 (force eggs), HR4=666 (signature), C11=True (protection trap). **Order matters** — disable protection (C11) *first*, *then* change HR0, else C15 (self-destruct) arms a 30-second countdown that dumps inventory to Zone 10. *Addresses are zero-indexed.* Investigate with Python + `pymodbus` (`read_holding_registers` , `read_coils`). Real ICS malware: **FrostyGoop** (2024).

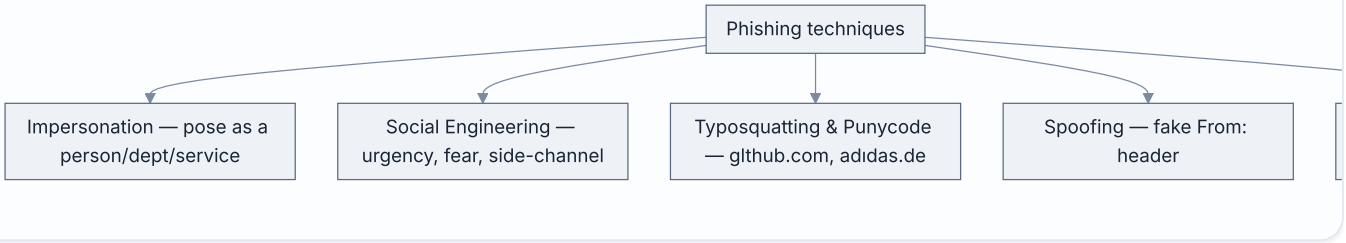
Phishing

The oldest trick, still the most effective — targeting *people*, the one thing technology can't patch. (See Humans as an Attack Vector.)

📘 **SPAM VS PHISHING**

Spam = bulk digital noise (quantity over precision; promotion, scams, clickbait). **Phishing** = a precision strike to deceive specific users (credential theft, malware delivery, data exfiltration, financial fraud).

Techniques — at least one is always present:



📘 **SPF · DKIM · DMARC**

Email-authentication checks in the `Authentication-Results` header. **SPF** lists servers allowed to send for a domain. **DKIM** adds a signature proving the message wasn't altered. **DMARC** uses both to decide what to do with fakes (spam/block). *SPF + DMARC both fail = strong spoofing sign.* Check `Return-Path` to spot the real sender and Punycode (ACE prefix).

Trending phishing moved from dropping malware to **stealing access**: abusing legitimate tools (Dropbox, Google Drive, OneDrive) to host lures → **fake login pages** → **side-channel communication** (SMS, WhatsApp, calls) to continue the con off-email.

20 · Quick Command Reference

A consolidated lookup card. See each chapter for context.

Linux essentials

Command	Does
`pwd` · `ls -al` · `cd ..`	Where am I · list all incl. hidden · up one
`cat` · `grep X file` · `find -name "*.txt"`	Read · search in files · find files
`cp` · `mv` · `rm -R` · `touch` · `mkdir`	Copy · move · remove dir · new file · new dir
`chmod +x s.sh` · `./s.sh`	Make executable · run
`ps aux` · `top` · `kill -9 PID`	Processes · live view · force kill
`ss -tunlp`	Listening ports (replaces `netstat`)
`wget URL` · `scp src user@ip:/dst`	Download · secure copy
`python3 -m http.server`	Serve current dir on :8000
`man cmd` · `cmd --help`	Documentation

Windows CMD / PowerShell

CMD	PowerShell	Does
`dir`	`Get-ChildItem`	List
`cd`	`Set-Location`	Change dir
`type`	`Get-Content`	Read file
`systeminfo`	`Get-ComputerInfo`	System info
`tasklist` / `taskkill /PID`	`Get-Process`	Processes
`ipconfig /all`	`Get-NetIPConfiguration`	Network config
`netstat -abon`	`Get-NetTCPConnection`	Connections

Offensive & analysis tools

Tool	One-liner
Nmap	`nmap -sV -p- IP` — service scan, all ports
gobuster	`gobuster -u URL -w wordlist.txt dir` — hidden dirs
curl	`curl -X POST -d "a=b" URL` · `-c/-b cookies.txt` · `-A agent`
John	`zip2john f.zip > h.txt; john --wordlist=rockyou.txt h.txt`
pdfcrack	`pdfcrack -f f.pdf -w rockyou.txt`
YARA	`yara -r rules.yar C:\` (`-s` show strings)
Zeek → RITA	`zeek readpcap c.pcap out/` → `rita import --logs out/ --database d` → `rita view d`
AWS CLI	`aws sts get-caller-identity` · `aws iam list-roles` · `aws s3api list-buckets`
Docker	`docker ps` · `docker exec -it name sh`
pymodbus	`client.read_holding_registers(address=0, count=1, slave=1)`

Splunk SPL / Kibana KQL

```
index=windows_sysmon ParentImage="*httpd.exe"
index=windows_apache_access (cmd.exe OR powershell OR "Invoke-Expression")
```

```
Source_ip : 238.163.231.224 AND UserName : Suleman
"United States" AND NOT ("Florida")
```

Common ports

21 FTP	**22 SSH**	**53 DNS**	**80 HTTP**	**443 HTTPS**	**445 SMB**	**502 Modbus**	**3306 MySQL**	**3389 RDP**

Nmap quick reference

Flag	Does
`nmap IP`	Scan top 1000 TCP ports
`-p-`	All 65535 TCP ports
`-p 22,80,443`	Specific ports
`-sV`	Service/version detection
`-sU`	UDP scan
`-sC` / `--script`	Default scripts / NSE scripts
`-O`	OS detection
`-T4`	Faster timing
`-vv`	Very verbose

DNS record types

Record	Resolves to
A	IPv4 address
AAAA	IPv6 address
CNAME	Another domain name (alias)
MX	Mail servers (with priority)
TXT	Free text (SPF, DKIM, verification)
NS	Authoritative nameservers

HTTP status codes

Range	Meaning	Common
1xx	Informational	100 Continue
2xx	Success	**200** OK · 201 Created
3xx	Redirection	301 Moved · 302 Found
4xx	Client error	400 · 401 · 403 · **404** · 405
5xx	Server error	500 · 503

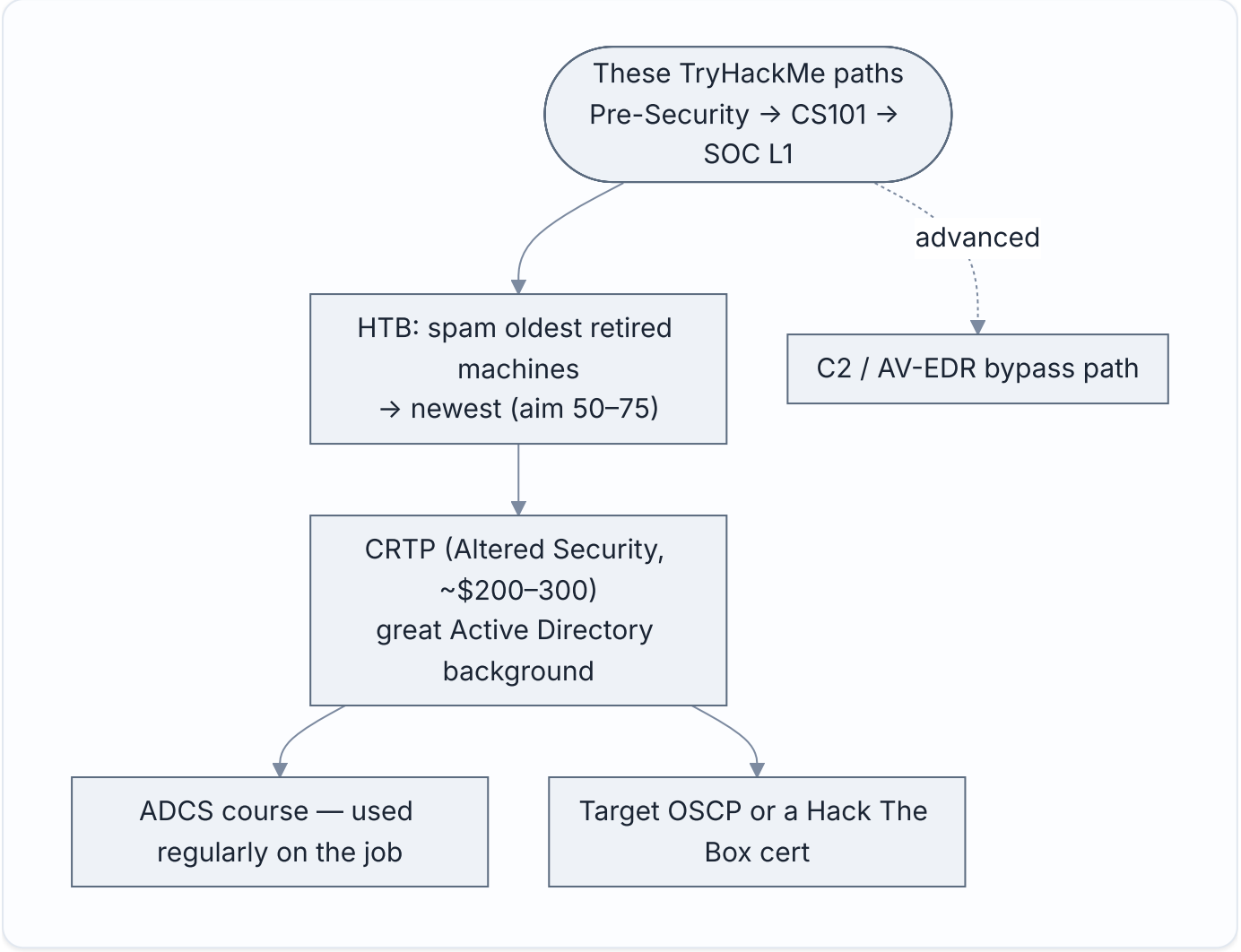
Framework cheat-sheet

Framework	One-line purpose
Cyber Kill Chain	7 linear stages of an intrusion (Lockheed Martin)
Unified Kill Chain	18 phases, non-linear, IN → THROUGH → OUT
MITRE ATT&CK	Knowledge base of adversary TTPs
MITRE D3FEND	Defensive counterpart to ATT&CK
Pyramid of Pain	Ranks indicators by how much they hurt the attacker
CIA Triad	Confidentiality · Integrity · Availability
Five Ws	Who / What / When / Where / Why — alert reports

21 · Your Learning Roadmap

Captured from your root "Where To Start" notes — practitioner advice on where to go after these foundations.

> 🔑 **THE CORE PHILOSOPHY**
>
> *"Honestly, spam as many CTF boxes as you can. It's less about memorising and more about knowing something exists."* — e.g. *"oh, 6379, I remember seeing that on a CTF"* → check your notes → it's Redis → look at Redis exploits. **And take good notes — they really help.** (The author still uses their OSCP notes.)



The cheaper path

1. **Hack The Box** — spam the oldest retired machines to newest. After ~50–75 you'll have a good grasp.

2. **CRTP** (Certified Red Team Professional, Altered Security) — gives a strong **Active Directory** background and reflects real day-job work. Their **ADCS** course is great too.
3. Then target **OSCP** or one of the Hack The Box certs.

Going deeper: C2 & modern defence evasion

For **C2 frameworks** or **bypassing modern defences (AV/EDR)**:

- **Maldev Academy** + **Matt Hand's book** (*Evading EDR*).
- **LitterBox** — self-learning tool: `github.com/BlackSnufkin/LitterBox`.

> 💡 **GLOSSARY FROM YOUR NOTES**
>
> - **Red Team** — offensive security.
> - **Blue Team** — cyber security & technology professionals protecting an information system from threats via defensive actions.
> - **SOC** — a team monitoring, preventing, detecting, investigating & responding to threats across a company's network/systems.
> - **XOR** — a binary operation for encryption/decryption; result is `1` if the two bits differ, `0` if the same.
> - **Container** — a package bundling code + dependencies so it runs reliably in any environment.

22 · The Noise

Everything below was **removed from the main handbook** during editing, kept here verbatim so you can see exactly what was trimmed and why. Nothing from your vault was deleted outright.

Duplicate files

Your vault contained many **exact duplicate notes** (identical content, two filenames — usually one with a hyphen, one without). The main handbook used one copy of each; these were the duplicates:

Kept	Exact duplicate removed
`1. TryHackMe - Linux CLI.md`	`1. TryHackMe Linux CLI.md` **and** `Red Team/TryHackMe Notes.md` (all three identical)
`2. TryHackMe - Splunk Intro.md`	`2. TryHackMe Splunk Intro.md`
`3. TryHackMe - AI In Security.md`	`3. TryHackMe AI In Security.md`
`4. TryHackMe - IDOR.md`	`4. TryHackMe IDOR.md`
`5. TryHackMe - Malware Analysis.md`	`5. TryHackMe Malware Analysis.md`
`6. TryHackMe - Network Discovery.md`	`6. TryHackMe Network Discovery.md`
`7. TryHackMe - Prompt Injection.md`	`7. TryHackMe Prompt Injection.md`
`8. TryHackMe - Passwords.md`	`8. TryHackMe Passwords.md`
`9. TryHackMe - SOC Alert Triaging.md`	`9. TryHackMe SOC Alert Triaging.md`
`Windows Command Line/Network Troubleshooting.md`	`Network Troubleshooting (1).md` (near-identical; only a trailing newline differed)
`Red Team/Where To Start Notes.md`	`Cybersecurity Learning/Where To Start Notes.md` (the second adds a few extra glossary call-outs, folded into the Roadmap)

Empty / placeholder notes

- `Welcome.md` — the default Obsidian vault note: *"This is your new vault. Make a note of something, [[Where To Start Notes]], or try the Importer! When you're ready, delete this note and make the vault your own."*
- `Cyber Defense Frameworks/Summit/Untitled.md` — empty file (no content).

Lab-specific flags, credentials & walkthrough plumbing

These are tied to one TryHackMe lab instance and have no reuse value, so they were kept out of the concept chapters:

- **Captured flags / answers:** `THM{SU_USER2}`, `su -root` (privilege-escalation quiz answer), the CyberChef "lock" gate answers (e.g. Username `Q290dG9uVGFpbA==` / Password `Iamsofluffy`; Gate 2–5 credentials), and recovered passwords from the cracking lab (`naughtylist`, `winter4ever`).
- **Hard-coded lab machine IPs:** `10.67.130.80`, `10.65.138.178`, `10.66.162.75:5001`, `10.65.131.10`, etc. — these change every time a room is deployed.
- **Side-Quest / bonus-mission hooks** and "click the SOC dashboard link below" instructions that only work inside the live TryHackMe environment.

Link lists & story framing

- **TryHackMe career-guide link dumps** from *Careers in Cyber* (e.g. *"Becoming a Cyber Security Analyst", "A Day in the Life of a SOC Analyst", success-story blog links*) — the role *content* was kept in Chapter 1; the marketing links were dropped.
- **"Advent of Cyber" narrative flavour** — the SOC-mas / King Malhare / Wareville / Evil Bunnies storyline framing. The *technical* substance of every day was kept; the festive story wrapper was trimmed for a reference handbook. (If you'd like the narrative restored, it's a one-line change in the source.)
- **Repeated "Learning Objectives" and room "Conclusion/Prerequisites" boilerplate** — these appeared at the top/bottom of almost every note; their unique content was merged into chapter intros rather than repeated 50+ times.

> 💡 **WANT ANY OF THIS BACK?**
>
> Everything here is a deliberate editorial call, not a deletion. Say the word and I can fold the Advent-of-Cyber storyline, the full career-link lists, or the lab flags back into the relevant chapters.